



One-pass online learning from data streams with unpredictable feature evolution

Peng Zhang^a, Hongpeng Yin^{a,*}, Han Zhou^{a,b}

^a The School of Automation, Chongqing University, Chongqing, 400044, China

^b The Department of New Networks, Peng Cheng Laboratory, Shenzhen, 518055, China

ARTICLE INFO

Keywords:

Unpredictable feature evolution

One-pass learning

Kernel-based online learning

Support vector selection strategy

ABSTRACT

In recent years, the field of online learning has gained significant attention, particularly in addressing the challenges posed by feature evolution in data streams, namely the disappearance of old features and the emergence of new ones. Many existing methods assume that feature evolution follows a predictable pattern, such as feature increment and specific retention of old features. However, in real-world scenarios, feature evolution is often unpredictable due to the uncertainty of feature disappearance and emergence. To address this challenge, this paper proposes a novel method called OLUFE for online learning from data streams with unpredictable feature evolution. OLUFE is a one-pass learning method where each instance appears only once, adhering to the evolving nature of data streams. It handles the dynamic evolution of the feature space with a kernel-based online learning method. Additionally, a projection-based support vector budget strategy is introduced to enhance the computational efficiency of the model. Theoretical analysis demonstrates that OLUFE achieves a sublinear regret $\mathcal{O}(\sqrt{T})$ under mild conditions. Extensive experiments on both synthetic and real-world datasets validate the effectiveness of OLUFE in handling data streams with unpredictable feature evolution.

1. Introduction

With the advent of the digital age, a large amount of data is generated, and these data often appear in the form of streams. Currently, learning with data streams has been extensively studied [1–3]. However, most existing methods assume that the feature space of each data stream is fixed. Unfortunately, in many real-world scenarios, these assumptions do not hold. The feature space of data streams often changes over time, with old features disappearing and new ones emerging [4,5]. Such data streams are referred to as feature evolution data streams. For example, in industrial monitoring systems, various sensors are used to monitor production activities, capturing signals such as temperature, humidity, pressure, flow, vibration, and so on. However, due to the need for dynamic adjustments based on production activities, sensors are added, removed, or replaced. This directly leads to the dynamic change of the feature space of the monitoring data, including the disappearance or emergence of features [6,7]. This requires the development of effective methods for handling disappearing and emerging features to adapt to feature evolution data streams.

Some existing methods, such as online convex optimization (OCO) [8] and extreme learning machine (ELM) [9], focus on adapting existing models to feature evolution data streams by learning from newly

emerged features. Learning with feature evolvable streams (FESL) [10] and prediction with unpredictable feature evolution (PUFE) [6] assume that when feature evolution occurs, there is an overlapping period during which both new and old features coexist. Fig. 1(a) and (b) illustrate the data streams for FESL and PUFE, respectively. They handle feature evolution by learning a mapping from the new feature space to the old one. In contrast, online learning with streaming features (OLSF) [11], one-pass incremental and decremental learning approach (OPID) [12], and generative learning with streaming capricious (GLSC) [13] do not assume the existence of the overlapping period. Instead, they learn by imposing constraints on feature evolution, such as ensuring that features increase monotonically, retaining certain old features, and representing relationships between features using a graph structure. These data streams are illustrated in Fig. 1(c), (d), and (e).

Although these methods have shown good performance in their respective feature evolution data stream settings, they have certain limitations. For instance, when features begin to evolve, the instances associated with new features are often limited, making it difficult for methods like OCO and ELM to train a robust model [14]. Furthermore, when features disappear in subsequent iterations, the learned patterns associated with these features are ignored, which limits the model's

* Corresponding author.

E-mail addresses: zhangpeng1917@163.com (P. Zhang), yinhongpeng@gmail.com (H. Yin), zhouhan1515@foxmail.com (H. Zhou).

<https://doi.org/10.1016/j.patcog.2025.112003>

Received 6 January 2025; Received in revised form 25 April 2025; Accepted 9 June 2025

Available online 5 July 2025

0031-3203/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

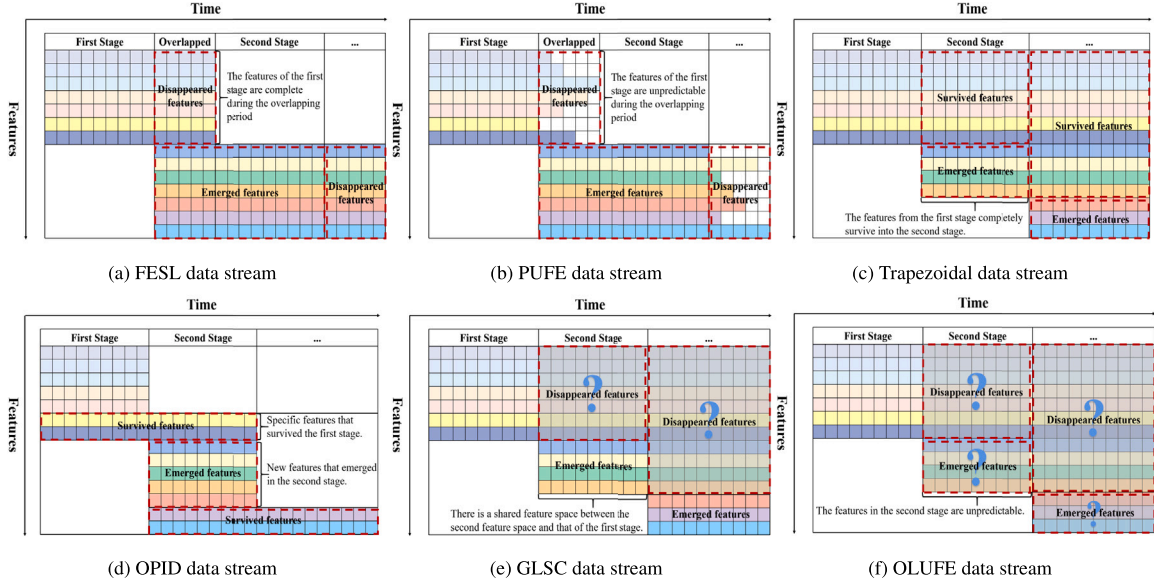


Fig. 1. Different scenarios of feature evolution in data streams.

Each square in the figure represents a feature, with squares of the same color denoting identical features. The feature spaces of the first and second stages correspond to the previous and current feature spaces, respectively. The ? symbol indicates an unpredictable feature state. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 1
Summary of related work.

Ref.	Prerequisites for feature dynamics	Mathematical expression
FESL [10]	Overlapping period with complete old features.	$\exists \tau \in [T], U_{\tau} = U_{old} \cup U_{new}$
PUFE [6]	Overlapping period with incomplete old features.	$\exists \tau \in [T], U_{\tau} = M U_{old} \cup U_{new}, M \sim \mathcal{R}$
OLSF [11]	Features are monotonically increasing.	$\forall t \in T, U_t \subseteq U_{t+1}$
OPID [12]	Old features disappear and survive in a predictable manner.	$\forall s > 1, U_s = A U_{old} \cup U_{new}, A \in \mathbb{R}^{d_s \times d_{old}}$
GLSC [13]	Relationships between features can be modeled graphically.	$\forall (x_i, x_j) \in U, G = (U, E)$
OLUFE (Ours)	No constraints (fully unpredictable).	$\forall s > 1, U_s \cap \bigcap_{i=1}^{s-1} U_i \in \{\emptyset, U_i, U_s, M U_i \mid M \sim \mathcal{R}\}$

T : Data stream sequence; U_{old} : Old feature set from the previous stage; U_{new} : Newly emerged feature set from the current stage; A : Known matrix; M : Random matrix ($\mathcal{R}$: Unpredictable mapping); G : Graph with vertices U (Feature set) and edges E (Relationships between features); s : Feature evolution scenario sequence.

predictive performance [15]. At the same time, In real-world data streams, the overlapping period may be very brief, making it challenging for FESL and PUF to effectively learn the mapping between the new and old feature spaces. Moreover, the assumptions made by OLSF, OPID, and GLSC may be strict, especially when the feature evolution in actual data streams is unpredictable. For example, in reactor monitoring within the chemical industry, when production switches from one chemical to another, the feature space of the monitoring data may change drastically. This unpredictability in feature evolution makes it impossible for the model to learn effectively.

To better reflect real-world scenarios, this paper focuses on data streams with unpredictable feature evolution, eliminating the assumptions about feature evolution made in prior works, as shown in Fig. 1(f). When feature evolution occurs, features from the previous feature space may arbitrarily disappear in the current stage. At the same time, the relationship between the newly emerged features in the current stage and those from the previous stage is also uncertain. This uncertainty makes feature evolution unpredictable. We have summarized the differences between this study and previous works in Table 1. A complete cycle consists of the previous and current stages. Since the entire process comprises continuous cycles, this study focuses on the scenarios within a single cycle.

This paper proposes a novel kernel-based online learning method (OLUFE) to address the challenges of online learning in data streams with unpredictable feature evolution. Specifically, OLUFE treats each feature variant as an independent feature set and employs a training and prediction strategy based on kernel functions and ensemble

methods. This strategy enables the model to effectively identify both disappearing and newly emerging features. Furthermore, a support vector selection strategy based on budget and projection methods is introduced to control the number of support vectors in the model. This strategy projects the information of newly added data into the subspace of existing support vectors, reducing redundant computations while retaining key information to the greatest extent, all while satisfying budget constraints. The effectiveness of the proposed method is demonstrated through experiments on feature evolution data streams across various scenarios. The main contributions of this paper are threefold:

- A more generalized feature evolution data stream learning problem is introduced, where the disappearance of old features and the emergence of new ones are both unpredictable. This avoids imposing constraints on feature dynamics, making it more aligned with real-world scenarios.
- A kernel-based online learning framework is designed that adaptively matches shared subspaces through kernel adjustment and integration strategies. This overcomes the limitations of traditional kernel methods in heterogeneous feature spaces while ensuring model scalability.
- A projection-based support vector budget strategy is proposed, maintaining a fixed budget by projecting removed support vector information into the current ones, thereby reducing redundant computations and enhancing both computational efficiency and model generalization.

- The performance upper bound of OLUFE is derived under mild conditions. Extensive experiments on public datasets and a real industrial process dataset validate the effectiveness of OLUFE.

In subsequent sections, a comprehensive overview of related work is provided in Section 2. The proposed approach is detailed in Section 3. The experimental setup and results analysis are presented in Section 4. Finally, conclusions are drawn based on the findings and contributions discussed in this paper in Section 5.

2. Related work

This work is primarily related to online learning with dynamic feature spaces and kernel-based online learning methods. Therefore, the literature on these two areas will be briefly reviewed.

2.1. Online learning with dynamic feature spaces

In recent years, online learning with dynamic feature spaces has garnered considerable attention, particularly in addressing the disappearance of old features and the emergence of new ones in data streams [13, 16]. Existing methods for online learning with dynamic feature spaces can be classified into two categories: those with overlapping periods and those without.

For online learning methods with overlapping periods, the foundational assumption is the existence of a period during the feature evolution process where old and new features coexist [6,10,17]. Hou et al. [10], building on the assumption that old and new features coexist and are complete during the overlap period, proposed the FESL method. This method aims to learn the mapping from the new feature space to the old feature space during the overlap period, adapting to the feature evolution data stream. However, the assumptions made by FESL may be overly stringent, as old features in actual data streams may not be complete during the overlap. Zhang [17] and Hou [6] conducted an in-depth exploration and extension of [10]. [17] proposed the Evolutionary Difference Minimization method (EDM) to learn the dynamically transformed feature space and data distribution. [6] introduced the PUF method to complete the incomplete old feature space through matrix completion. However, PUF may face challenges in accurately predicting during the overlapping process.

Diverging from the above studies, some research focuses on online learning methods without overlapping periods. Zhang et al. [11] introduced a trapezoidal data stream, wherein the features of new samples are consistently equal to or greater than the features of old samples. However, new features in data streams may not consistently exhibit incremental trends. Hou et al. [12] proposed the OPID method to compress the disappeared old feature information into the surviving features and learn them together with the new features. Similarly, Ye et al. [18] use optimal transfer to learn mapping functions from two different feature spaces. However, [12,18] does not consider streaming mode but uses batch mode. He et al. [12] use graph methods for learning. However, in many real-life scenarios, the relationship between features may not be represented by a graph structure. To be closer to the real scenario, Beyazit et al. [19] and You et al. [20] refrain from making dynamic assumptions about the old and new features in the data stream. [19] classifies instances by learning two classifiers simultaneously at the feature space level and instance level. However, the performance of classifiers is significantly affected by feature completeness. [20] proposed online learning from incomplete and imbalanced data streams (OLI²DS) method to address the challenges of learning from incomplete and imbalanced data streams, using the empirical risk minimization principle and a dynamic cost strategy. However, OLI²DS may be more suitable for addressing concept drift and class imbalance problems in data streams.

2.2. Kernel-based online learning

Data within data streams often exhibit complexity and may lack fixed linear or nonlinear relationships. When grappling with data showcasing nonlinear relationships, the adoption of nonlinear learning methods becomes imperative [21–23]. Kernel-based online learning stands out as a pivotal algorithm in the realm of nonlinear learning [24, 25]. Diverging from traditional batch kernel methods, kernel-based online learning algorithms leverage kernel functions to map input data to a higher-dimensional feature space, enabling learning from individual data instances [26–28]. This characteristic renders kernel-based online learning more efficient and scalable, particularly suited for large-scale data stream prediction. However, despite the enhancements these methods bring to the model's classification performance for data streams, they grapple with the challenge known as the kernelization curse, that is, the infinite increase in the number of support vectors [29].

To counter this challenge, several studies have proposed budget-based kernel online learning algorithms [30]. Cerone et al. [31] establish a trade-off between model complexity and accuracy by reducing the number of basis functions employed, while also calculating the bound on the resulting approximation error. Le et al. [32] employed sparsification methods to constrain the model size, maintaining computational efficiency at an acceptable level. Although these methods address the challenge of an infinitely increasing number of support vectors, their oversimplified strategies may inadvertently remove critical information, thereby hindering the model from achieving optimal performance.

In a bid to further enhance model performance, some methods attempt to seek the optimal support vector set based on a budget. Liao et al. [33] designed a budget maintenance mechanism that removes half of the samples and projects the discarded samples into a hypothesis space formed by the remaining samples. Wang et al. [9] maintains the number of cores through core fusion methods. Qi et al. [34] proposed a fusion kernel strategy that captures consistency information among kernels by integrating basis kernel tensors into fusion kernels via tensor operations. Li et al. [35] employs regret analysis to determine the budget size, thereby achieving a balance between model complexity and regret. However, none of these algorithms address the issue of dynamic changes in feature space. Shen et al. [36] improved computational efficiency through a scalable online multi-kernel learning method based on kernel dictionaries. In addition, several approaches have adopted distributed collaborative training to enhance efficiency [37,38]. Shen et al. [37] proposed a distributed quantization algorithm, where each node shares only its locally estimated gradient of the loss function with neighboring nodes. Kwon et al. [38] reduced computational overhead by employing intermittent model updates and local gradient quantization.

3. Proposed algorithm

In this section, the underlying rationale behind the proposed OLUFE method is initially presented. Subsequently, the corresponding theoretical analysis of the proposed approach is delved into.

3.1. Problem setting

Let $\mathcal{D}$ be the set of instances arriving in sequence $\{(\mathbf{x}_t, \mathbf{y}_t) \mid t = 1, \dots, T\}$, where $\mathbf{x}_t \in \mathbb{R}^{d_t}$ is the feature vector of the instance at time t , and $\mathbb{R}^{d_t}$ is the feature space with feature dimension d_t at time t . $\mathbf{y}_t \in \{-1, +1\}$ is the binary label of the instance. $\mathbf{U}_s$ is the feature space in the data stream after the s th feature evolution, defined as $\mathbf{U}_s = \mathbb{R}^{d_s}$, where d_s is the feature dimension of the data stream after the s th evolution. Feature evolution is unpredictable. When it occurs, the dimension of the feature space may change, i.e., $\mathbb{R}^{d_t} \neq \mathbb{R}^{d_{t+1}}$, resulting in the emergence of the new feature space $\mathbf{U}_s$. The relationship

between U_s and the previous feature spaces $U_1, U_2, \dots, U_{s-1}$ is arbitrary, namely, $U_s \cap U_i$ for $i = 1, 2, \dots, s-1$ being uncertain. Specifically, several scenarios may occur: (i) all features in U_i survive in U_s without any disappearance, i.e., $U_s \cap U_i = U_i$. (ii) no new features emerge in U_s , i.e., $U_s \cap U_i = U_s$. (iii) some features from U_i disappear in U_s while new features emerge, i.e., $U_s \cap U_i \neq \emptyset$, $U_i \not\subseteq U_s$, and $U_s \not\subseteq U_i$. (iv) all features from U_i disappear, and U_s consists entirely of newly emerged features, i.e., $U_s \cap U_i = \emptyset$. The task is that, at each iteration, the learner observes a single instance $\mathbf{x}_t \in D$ and predicts whether its label y_t is -1 or $+1$. After the prediction is made, the true label of $\mathbf{x}_t$ is revealed, and the algorithm incurs an instantaneous loss, reflecting the difference between the predicted and true labels. The goal of the algorithm is to minimize the prediction error of the current instance $(\mathbf{x}_t, y_t)$ at each time step:

$$\min_f \ell(f(\mathbf{x}_t); (\mathbf{x}_t, y_t)), \quad (1)$$

where ℓ is a convex function that measures the loss between the model prediction $f(\mathbf{x}_t)$ and the true label y_t .

The kernel-based online gradient descent method is employed to update the learner f in order to achieve this goal:

$$\begin{aligned} f_{t+1} &= f_t - \eta y_t \nabla \ell(f(\mathbf{x}_t), (\mathbf{x}_t, y_t)) K(\mathbf{x}_t, \cdot) \\ &= f_t + \alpha_t K(\mathbf{x}_t, \cdot) \end{aligned} \quad (2)$$

where $\eta > 0$ is the learning rate that controls the step size of each update. α_t is the coefficient for $\mathbf{x}_t$. $\nabla \ell$ is the gradient of the loss function ℓ . $K(\cdot, \cdot)$ is a kernel function in reproducing kernel Hilbert space (RKHS) $\mathcal{H}$, which is used to compute the similarity between the feature vector of the current instance and the support vectors.

3.2. The basic model under evolving features

During the iterative process, whenever a new instance is received, the model matches its feature subset with the existing feature scenarios. If the feature subset does not align with any existing scenario, i.e., $U_i \not\subseteq U_s$ and $U_s \not\subseteq U_i$, a new feature scenario is created. The model initializes the support vectors and their corresponding weights to accommodate the newly emerged features. Concurrently, for feature scenarios that remain unused over extended periods, a dynamic weight adjustment mechanism is applied to reduce their influence, thereby effectively addressing disappeared features.

Assume there are S feature scenes, and a prediction sub-model $f_{s,t}(\cdot)$ is learned in each scene. Each $f_{s,t}(\cdot)$ is updated by Eq. (2), where $s \in \{1, 2, \dots, S\}$. The model makes ensemble predictions by combining the outputs of all sub-models:

$$\begin{aligned} F_t(\cdot) &= f_{1,t}(\cdot) + f_{2,t}(\cdot) + \dots + f_{S,t}(\cdot) \\ &= \sum_{i=1}^{n_1} \omega_1 \alpha_i K(\mathbf{x}_i, \cdot) + \sum_{i=1}^{n_2} \omega_2 \alpha_i K(\mathbf{x}_i, \cdot) + \dots + \sum_{i=1}^{n_S} \omega_S \alpha_i K(\mathbf{x}_i, \cdot) \\ &= \sum_{s=1}^S \sum_{i=1}^{n_s} \omega_s \alpha_i K(\mathbf{x}_i, \cdot), \end{aligned} \quad (3)$$

where ω_s is the weight parameter of the s th sub-model, controlling the importance of the corresponding sub-model. n_s is the number of support vectors in the s th sub-model.

It is important to note that due to the heterogeneity between instances $\mathbf{x}_i$ and $\mathbf{x}_j$, the standard kernel function calculation method has limitations. To address this, the kernel function is adjusted. Specifically, when shared features exist between the instances $\mathbf{x}_i$ and $\mathbf{x}_j$, the kernel value $K(\mathbf{x}_i^c, \mathbf{x}_j^c)$ is computed within the shared feature subspace, i.e., $K(\mathbf{x}_i^c, \mathbf{x}_j^c) = \exp\left(-\frac{\|\mathbf{x}_i^c - \mathbf{x}_j^c\|^2}{2\sigma^2}\right)$, where $\mathbf{x}_i^c, \mathbf{x}_j^c \in \mathbb{R}^{d_i} \cap \mathbb{R}^{d_j}$. If no shared features are present between the instances $\mathbf{x}_i$ and $\mathbf{x}_j$, i.e., $d_i \cap d_j = \emptyset$, the kernel function returns a zero matrix.

3.3. The updating strategy

Once receiving a new labeled instance $\mathbf{x}_t$ at time t , the model needs to include the instance as a support vector for updating

$$F_{t+1}(\cdot) \leftarrow F_t(\cdot) + \omega_{s_t} \alpha_t K(\mathbf{x}_t, \cdot) \quad (4)$$

where s_t denotes the feature scenario at time t . Unfortunately, using all instances as support vectors can lead to the kernel curse, where the computational complexity increases with time. Therefore, two strategies are proposed to enhance computational efficiency: support vector selection and projection-based budget.

3.3.1. Support vector selection

The support vector selection strategy aims to select only informative instances as support vectors. The key idea is that learning only instances that provide the greatest insight into the underlying data distribution can boost learning performance. Let $\mathbb{V}_s$ be the support vector set for the s feature set scenario at time t . This set is updated as follows:

$$\mathbb{V}_s \leftarrow \begin{cases} \mathbb{V}_s \cup \{\mathbf{x}_t\}, & |y_t f_{s,t}(\mathbf{x}_t)| \leq \rho; \\ \mathbb{V}_s, & \text{otherwise.} \end{cases} \quad (5)$$

where $\rho > 0$ is a parameter that controls the sensitivity of the selection strategy. The term $|y_t f_{s,t}(\mathbf{x}_t)|$ represents the confidence.

When $y_t f_{s,t}(\mathbf{x}_t) > 0$, the instance is correctly classified, and the larger its absolute value, the higher the confidence. Conversely, when $y_t f_{s,t}(\mathbf{x}_t) < 0$, the instance is misclassified, and the larger its absolute value, the higher the incorrect confidence. Therefore, $|y_t f_{s,t}(\mathbf{x}_t)|$ can be used to estimate the uncertainty of each incoming instance. Instances with uncertainty values below a predefined threshold ρ are selected, as they lie closer to the decision boundary and thus provide more informative feedback for model updates.

The advantages of this support vector selection strategy are twofold: (1) speed up the learning procedure by discarding some less informative instances; (2) maximize the learning performance by focusing on more informative instances and help alleviate the bias of the model towards the majority class.

However, although this strategy can significantly reduce the size of support vectors, as more instances are received over time, the number of support vectors can increase to infinity. To address this issue, it is necessary to constrain the number of support vectors.

3.3.2. Projection based budget

To constrain the number of support vectors, it is straightforward to maintain a fixed size for the set and remove the oldest support vector when necessary, once the size of the support vector set exceeds B . This prevents the size and computational complexity of the sub-models from growing without bounds as the amount of data increases, thereby ensuring the scalability of the model. However, this may also discard historical information, hindering the prediction model from achieving optimal performance.

To address this issue, the projection strategy is employed. The goal is to find a scalar Δ such that the information of the removed support vector $\mathbf{x}_r$ can be embedded into the current support vector $\mathbf{x}_t \in \mathbb{R}^{d_t}$. The procedure of finding the optimal Δ can be formulated as a convex optimization problem that minimizes the projection error:

$$\min_{\Delta} \left\| \omega_r \alpha_r K(\mathbf{x}_r^c, \mathbf{x}_t^c) - \omega_S \Delta K(\mathbf{x}_t, \mathbf{x}_t) \right\|_{\mathcal{H}}^2. \quad (6)$$

This equation has the following closed solution:

$$\Delta = \frac{\omega_r \alpha_r K(\mathbf{x}_r^c, \mathbf{x}_t^c)}{\omega_S K(\mathbf{x}_t, \mathbf{x}_t)}, \quad (7)$$

where $\mathbf{x}_r^c$ denotes the removed instance containing only the common features shared with the t th instance. To embed the information of $\mathbf{x}_r$ into $\mathbf{x}_t$, the coefficient for $\mathbf{x}_t$ is:

$$\alpha_t = -\eta_t y_t \ell(F_t(\mathbf{x}_t)) + \Delta. \quad (8)$$

The remaining issue is finding the best candidate for projection among the support vector set. To simplify the process and make it more efficient, this paper uses a simplified solution that always projects the smallest support vector. This approach can be described as follows:

$$r = \min_r (\omega_r \alpha_r K(\mathbf{x}_r, \mathbf{x}_t))^2. \quad (9)$$

Here, r refers to the index of the support vector with the smallest value of, which is selected for projection.

By always selecting the smallest support vector for projection, the approach provides a simplified solution that reduces computational complexity while still preserving important historical information.

3.3.3. The overall updating rules

Receiving a new labeled instance $\mathbf{x}_t$ at time t , the model makes a prediction as:

$$\hat{y}_t = \text{sign}(F_t(\mathbf{x}_t)). \quad (10)$$

According to this prediction, the model suffers a loss:

$$\ell(F_t; (\mathbf{x}_t, \mathbf{y}_t)) = \max(0, 1 - \mathbf{y}_t F_t(\mathbf{x}_t)) \quad (11)$$

Then, the parameter of sub-models $f_{s,t}, s \in [S]$ need to be adjusted as:

$$\omega_s \leftarrow \begin{cases} \lambda \omega_s + (1 - \lambda), & \ell(f_{s,t}; (\mathbf{x}_t, \mathbf{y}_t)) = 0; \\ \lambda \omega_s, & \text{otherwise.} \end{cases} \quad (12)$$

where λ is the parameter that controls the rate of weight decay, with $\lambda \in (0, 1]$.

The core idea behind the dynamic weight adjustment mechanism is to assign smaller parameters to sub-models with higher misclassification errors. This mechanism has two advantages: (1) By focusing more on accurate sub-models, it maximizes learning accuracy. (2) Sub-models that remain unused for an extended period (such as those corresponding to disappeared feature scenarios) experience a decay in weight due to the continuous decay factor λ , eventually driving the weight towards zero and implicitly eliminating them. This ensures that the number of active sub-models in the model remains limited, preventing uncontrolled growth in model size and computational complexity, thereby maintaining the scalability of the model.

If the instance $\mathbf{x}_t$ is not selected by the rule Eq. (5), only the parameter need to be adjusted:

$$F_{t+1}(\cdot) \leftarrow \sum_{s=1}^S \sum_{\mathbf{x}_i \in \mathbb{V}_s} \omega_s \alpha_i K(\mathbf{x}_i, \cdot) \quad (13)$$

If the instance $\mathbf{x}_t$ is selected by the rule in Eq. (5) and the size of the support vector set does not exceed B , then the prediction model is updated by including the new support vector:

$$\begin{aligned} \mathbb{V}_s(\cdot) &\leftarrow \mathbb{V}_s \cup \{\mathbf{x}_t\}, \mathbf{x}_t \in \mathbb{R}^{d_s}; \\ \alpha_{t+1} &\leftarrow \eta \mathbf{y}_t; \\ F_{t+1}(\cdot) &\leftarrow \sum_{s=1}^S \sum_{\mathbf{x}_i \in \mathbb{V}_s} \omega_s \alpha_i K(\mathbf{x}_i, \cdot) \end{aligned} \quad (14)$$

The procedure of the proposed method is summarized in Algorithm 1.

If the instance $\mathbf{x}_t$ is selected by the rule in Eq. (5) but the size of the support vector set exceeds B , then the inclusion of new support vectors and the removal of the smallest one are required:

$$\begin{aligned} \mathbb{V}_s(\cdot) &\leftarrow \mathbb{V}_s \cup \{\mathbf{x}_t\}, \mathbf{x}_t \in \mathbb{R}^{d_s}; \\ \mathbb{V}_{s_r}(\cdot) &\leftarrow \mathbb{V}_{s_r} \setminus \{\mathbf{x}_r\}, \mathbf{x}_r \in \mathbb{R}^{d_{s_r}}; \\ \alpha_{t+1} &\leftarrow \eta \mathbf{y}_t + \Delta; \\ F_{t+1}(\cdot) &\leftarrow \sum_{s=1}^S \sum_{\mathbf{x}_i \in \mathbb{V}_s} \omega_s \alpha_i K(\mathbf{x}_i, \cdot) \end{aligned} \quad (15)$$

Algorithm 1 Online learning model under feature evolution

Input: Data stream $\mathcal{D}$, parameters λ and ρ , learning rate α_t , buffer size B .

```

1: for  $t = [1, T]$  do
2:   Receive instances  $\mathbf{x}_t \in \mathbb{R}^{d_t}$  and make predictions  $\hat{y}_t$ .
3:   Receive the true label  $\mathbf{y}_t$  and incur the loss by Eq. (11).
4:   Adjust the parameter  $\omega_s, s \in [S]$ .
5:   if  $|\mathbf{y}_t f_{s,t-1}(\mathbf{x}_t)| \leq \rho$  then
6:     if  $|\mathbb{V}_s| < B$  then
7:       Update model by Eq. (14)
8:     else
9:       Update model by Eq. (15).
10:    end if
11:  else
12:    Update model by Eq. (13).
13:  end if
14: end for

```

3.4. Theoretical analysis

This section explores the performance guarantees provided by the proposed OLUFE method. Firstly, the upper bound of the cumulative loss is discussed in an unconstrained case where the budget size approaches infinity, i.e. $B \rightarrow +\infty$. Then, the cumulative loss with respect to the parameter ρ is discussed. Finally, the cumulative loss is considered when some support vectors are removed.

Let the function $\mathcal{G}(\mathbf{x}_t)$ represent a hypothesis in $\mathcal{H}$. The squared norm of $\mathcal{G}$ in $\mathcal{H}$, denoted as $\|\mathcal{G}\|_{\mathcal{H}}^2$, measures the length or magnitude of the hypothesis $\mathcal{G}$ in this space. T is the size of the data stream. For the sake of analytical convenience, the case where $\lambda = 1$ is considered. Based on the above definitions, the bound for the cumulative loss can be established.

Theorem 1. Let $F_t, t \in [T]$ be a sequence of hypotheses generated by the updating rule in Eq. (3). Let $\mathbb{V}_T = \bigcup_{s=1}^S \mathbb{V}_s$ be the selected support vector set generated by Eq. (5) and $|\mathbb{V}_T| < B$. Assuming $K(\mathbf{x}_i, \mathbf{x}_j) \leq 1$ and the loss function $\ell(F_t(\mathbf{x}_t))$ is L -Lipschitz, then for any $\mathcal{G} \in \mathcal{H}$, the cumulative loss of the model F can be bounded as follows when :

$$\sum_{\mathbf{x}_t \in \mathbb{V}_T} \ell(F_t(\mathbf{x}_t)) \leq \sum_{\mathbf{x}_t \in \mathbb{V}_T} \ell(\mathcal{G}(\mathbf{x}_t)) + \frac{\|\mathcal{G}\|_{\mathcal{H}}^2}{2\eta} + \eta L^2 V_T \quad (16)$$

Let $\eta_t = \frac{1}{\sqrt{t}}$, then the algorithm achieves a sublinear regret $\mathcal{O}(\sqrt{T})$:

$$\sum_{\mathbf{x}_t \in \mathbb{V}_T} \ell(F_t(\mathbf{x}_t)) - \sum_{\mathbf{x}_t \in \mathbb{V}_T} \ell(\mathcal{G}(\mathbf{x}_t)) \leq \mathcal{O}(\sqrt{T}) \quad (17)$$

The Proof for Theorem 1. Following the update rules in Eqs. (13) and (14), and recalling $\alpha_t = -\eta \mathbf{y}_t \nabla \ell(F_t(\mathbf{x}_t))$, the equation can be expressed as follows:

$$\begin{aligned} \|F_t - \mathcal{G}\|_{\mathcal{H}}^2 - \|F_{t+1} - \mathcal{G}\|_{\mathcal{H}}^2 &= \|F_t - \mathcal{G}\|_{\mathcal{H}}^2 - \|F_t - \mathcal{G} + \alpha_t K(\mathbf{x}_t, \cdot)\|_{\mathcal{H}}^2 \\ &= \|2\eta \mathbf{y}_t \nabla \ell(F_t(\mathbf{x}_t)) \langle K(\mathbf{x}_t, \cdot), F_t - \mathcal{G} \rangle\|_{\mathcal{H}}^2 - \|\eta \mathbf{y}_t \nabla \ell(F_t(\mathbf{x}_t))^2 K(\mathbf{x}_t, \mathbf{x}_t)\|_{\mathcal{H}}^2. \end{aligned} \quad (18)$$

By utilizing the reproducing property of $\mathcal{H}$, it can be observed that $\langle k(\mathbf{x}_t, \cdot), F \rangle = F(\mathbf{x}_t)$. Recalling $\nabla \ell \leq L$ and $K(\mathbf{x}_i, \mathbf{x}_j) \leq 1$, the above formulation can be expressed as:

$$\nabla \ell(F_t) [F_t(\mathbf{x}_t) - \mathcal{G}(\mathbf{x}_t)] \leq \frac{\|F_t - \mathcal{G}\|_{\mathcal{H}}^2 - \|F_{t+1} - \mathcal{G}\|_{\mathcal{H}}^2}{2\eta} + \frac{\eta L^2}{2} \quad (19)$$

Due to the convexity of the loss function, it is obtained:

$$\ell(F_t(\mathbf{x}_t)) - \ell(\mathcal{G}(\mathbf{x}_t)) \leq \nabla \ell(F_t) [F_t(\mathbf{x}_t) - \mathcal{G}(\mathbf{x}_t)] \quad (20)$$

The combination of the above two inequalities yields:

$$\ell(F_t(\mathbf{x}_t)) - \ell(\mathcal{G}(\mathbf{x}_t)) \leq \frac{\|F_t - \mathcal{G}\|_{\mathcal{H}}^2 - \|F_{t+1} - \mathcal{G}\|_{\mathcal{H}}^2}{2\eta} + \frac{\eta L^2}{2} \quad (21)$$

Summing the derived inequality over $t = 1, \dots, V_T$, the following is obtained:

$$\sum_{\mathbf{x}_i \in \mathbb{V}_T} \ell(F_i(\mathbf{x}_i)) - \sum_{\mathbf{x}_i \in \mathbb{V}_T} \ell(G(\mathbf{x}_i)) \leq \frac{\|G\|_H^2}{2\eta} + \eta L^2 V_T \quad (22)$$

Rearranging the above inequality completes the proof.

Since V_T is the selected support vector set generated by Eq. (5), the cumulative loss is influenced by ρ , as concluded in the following corollary.

Corollary 1. *Under the assumptions and conclusions of Theorem 1, let the confidence of the instances in the data stream, $|y_{t,f_{s,t}}(\mathbf{x}_i)|$, follow the probability density function $p(z)$, where $z = |y_{t,f_{s,t}}(\mathbf{x}_i)|$. The cumulative loss of the model F can be further expressed as:*

$$\sum_{\mathbf{x}_i \in \mathbb{V}_T} \ell(F_i(\mathbf{x}_i)) \leq \sum_{\mathbf{x}_i \in \mathbb{V}_T} \ell(G(\mathbf{x}_i)) + \frac{\|G\|_H^2}{2\eta} + \eta L^2 T \int_0^\rho p(z) dz \quad (23)$$

The Proof for Corollary 1. Since $\mathbb{V}_T$ is the selected support vector set generated by Eq. (5), the probability that each instance is selected is given by:

$$P(z \leq \rho) = \int_0^\rho p(z) dz \quad (24)$$

Consequently, the expected of V_T can be expressed as:

$$V_T = T \int_0^\rho p(z) dz \quad (25)$$

Based on Eq. (22), the following expression can be obtained:

$$\sum_{\mathbf{x}_i \in \mathbb{V}_T} \ell(F_i(\mathbf{x}_i)) - \sum_{\mathbf{x}_i \in \mathbb{V}_T} \ell(G(\mathbf{x}_i)) \leq \frac{\|G\|_H^2}{2\eta} + \eta L^2 T \int_0^\rho p(z) dz \quad (26)$$

Next, the cumulative loss is quantified when the projection-based budget is used, as concluded in the following theorem.

Theorem 2. *Let $F_t, t \in [T]$ be a sequence of hypotheses generated by the updating rule in Eq. (3). Let $\mathbb{V}_T = \bigcup_{s=1}^S \mathbb{V}_s$ be the selected support vector set and $\mathbb{V}_R$ be the removed support vector set generated by Eq. (15). Assuming $K(\mathbf{x}_i, \mathbf{x}_j) \leq 1$ and the loss function $\ell(F_t(\mathbf{x}_i))$ is L -Lipschitz, then for any $G \in \mathcal{H}$, the cumulative loss of the model F can be quantified as:*

$$\begin{aligned} \sum_{\mathbf{x}_i \in \mathbb{V}_T \cup \mathbb{V}_R} \ell(F_i(\mathbf{x}_i)) &\leq \sum_{\mathbf{x}_i \in \mathbb{V}_T \cup \mathbb{V}_R} \ell(G(\mathbf{x}_i)) + \frac{\|G\|_H^2}{2\eta} + 2LV_R \|G\|_H \\ &\quad + \frac{9\eta L^2 (V_T + V_R)}{2} \end{aligned} \quad (27)$$

Let $\eta_t = \frac{1}{\sqrt{t}}$, the algorithm still achieves a sublinear regret $\mathcal{O}(\sqrt{T})$:

$$\sum_{\mathbf{x}_i \in \mathbb{V}_T \cup \mathbb{V}_R} \ell(F_i(\mathbf{x}_i)) - \sum_{\mathbf{x}_i \in \mathbb{V}_T \cup \mathbb{V}_R} \ell(G(\mathbf{x}_i)) \leq \mathcal{O}(\sqrt{T}) \quad (28)$$

The Proof for Theorem 2. Following the update rules in Eqs. (13) and (15), with $\alpha_t = -\eta y_t \nabla \ell(F_t(\mathbf{x}_i))$, the equation is obtained:

$$\begin{aligned} &\|F_t - G\|_H^2 - \|F_{t+1} - G\|_H^2 \\ &= \|F_t - G\|_H^2 - \|F_t - G + (\alpha_t + \Delta)k(\mathbf{x}_t, \cdot) - \alpha_r K(\mathbf{x}_r, \cdot)\|_H^2 \\ &= -\langle 2(\alpha_t + \Delta)k(\mathbf{x}_t, \cdot) - 2\alpha_r K(\mathbf{x}_r, \cdot), F_t - G \rangle \\ &\quad - \|(\alpha_t + \Delta)k(\mathbf{x}_t, \cdot) - \alpha_r K(\mathbf{x}_r, \cdot)\|_H^2 \end{aligned} \quad (29)$$

By utilizing the reproducing property of $\mathcal{H}$, it can be observed that $\langle k(\mathbf{x}_i, \cdot), F \rangle = F(\mathbf{x}_i)$. Recalling $\alpha_t = -\eta y_t \nabla \ell(F_t(\mathbf{x}_i))$, the equation is derived as follows:

$$\begin{aligned} &\|F_t - G\|_H^2 - \|F_{t+1} - G\|_H^2 = 2\eta \nabla \ell(F_t(\mathbf{x}_i)) [F_t(\mathbf{x}_i) - G(\mathbf{x}_i)] \\ &\quad + 2\eta \nabla \ell(F_r(\mathbf{x}_r)) K(\mathbf{x}_r^c, \mathbf{x}_t^c) [F_t(\mathbf{x}_i) - G(\mathbf{x}_i)] \\ &\quad - 2\eta \nabla \ell(F_r(\mathbf{x}_r)) [F_t(\mathbf{x}_r) - G(\mathbf{x}_r)] \\ &\quad - \|(\alpha_t + \Delta)k(\mathbf{x}_t, \cdot) - \alpha_r K(\mathbf{x}_r, \cdot)\|_H^2 \end{aligned} \quad (30)$$

Table 2

The fifteen datasets used in the experiments.

Dataset	#Instances	#Feat.1	#Feat.2
HTRU2	17 898	8	14
a8a	32 561	123	200
ada	4147	48	37
biodeg	1055	41	60
credit_g	1000	20	14
german	1000	59	41
ijcnn1	49 990	22	40
mammography	11 183	6	5
mc1	9466	38	20
musk2	6598	166	100
mw1	403	37	51
oil_spill	937	49	30
pc2	5589	36	22
svmguide3	1284	22	15
HRS	47 929	120	40

Use $\nabla \ell \leq L$ and $K(\mathbf{x}_i, \mathbf{x}_j) \leq 1$ and rearrange this equation:

$$\begin{aligned} \nabla \ell(F_r(\mathbf{x}_r)) [F_t(\mathbf{x}_r) - G(\mathbf{x}_r)] &\leq \frac{\|F_t - G\|_H^2 - \|F_{t+1} - G\|_H^2}{2\eta} \\ &\quad + \nabla \ell(F_r(\mathbf{x}_r)) [F_t(\mathbf{x}_r) - G(\mathbf{x}_r)] \\ &\quad + \|(\alpha_t + \Delta)k(\mathbf{x}_t, \cdot) - \alpha_r K(\mathbf{x}_r, \cdot)\|_H^2 \\ &\leq \frac{\|F_t - G\|_H^2 - \|F_{t+1} - G\|_H^2}{2\eta} + 2L \|G\|_H \\ &\quad + \frac{9\eta L^2}{2} \end{aligned} \quad (31)$$

Using the convexity of the loss function and summing the derived inequality over $t = 1, \dots, V_T + V_R$, the following expression can be obtained:

$$\begin{aligned} \sum_{\mathbf{x}_i \in \mathbb{V}_T \cup \mathbb{V}_R} \ell(F_i(\mathbf{x}_i)) - \sum_{\mathbf{x}_i \in \mathbb{V}_T \cup \mathbb{V}_R} \ell(G(\mathbf{x}_i)) &\leq \frac{\|G\|_H^2}{2\eta} + 2LV_R \|G\|_H \\ &\quad + \frac{9\eta L^2 (V_T + V_R)}{2} \end{aligned} \quad (32)$$

Rearranging the above inequality completes the proof.

4. Experiments

This section provides a comprehensive overview of the experiments.

4.1. Datasets and general settings

A total of 14 UCI datasets and 1 real-world dataset are used to evaluate the performance of the proposed method in this paper, as shown in Table 2. All UCI datasets are freely downloaded from the website: <http://archive.ics.uci.edu/ml>. These UCI datasets cover a wide range of domains. However, since these UCI datasets each contain only one original feature space, a new feature space is first generated for these datasets to simulate feature evolution data streams under different scenarios. To achieve this, the method described in [10] is applied. Specifically, a Gaussian random matrix is used to map the feature space of the original dataset D_{ori} to another feature space, generating a new dataset D_{new} in the transformed feature space. The randomness of the mapping matrix makes the relationship between the original feature space and the newly generated feature space unpredictable. It is worth noting that no assumptions are involved in this process. The feature space of D_{ori} is referred to as the previous feature space, denoted as P , while the feature space of D_{new} is referred to as the current feature space, denoted as C .

In addition to the synthetic data, experiments are also conducted on a real industrial process dataset (Hot Roll of Steel, HRS). This dataset consists of the operating status data from 294 rollers. The sampling frequency of the operating status data is 2 Hz, and the data spans half

Table 3
The 40 extracted features from roller data.

Number	Feature description	Number	Feature description
1	Mean	21	Kurtosis
2	Standard deviation	22	Skewness
3	Variance	23	Margin factor
4	Median	24	Shape factor
5	Difference mean	25	Impulse factor
6	Difference standard deviation	26	Maximum power spectral density frequency
7	Mean frequency	27	Spectral entropy
8	Peak value	28	Central frequency
9	Peak-to-Peak value	29	Frequency variance
10	Number of peaks	30	Signal entropy
11	Peak positions	31	Logarithmic energy
12	Peak density	32	Signal complexity
13	Peak factor	33	Wavelet packet feature 01
14	Signal range	34	Wavelet packet feature 02
15	Root Mean Square (RMS)	35	Wavelet packet feature 03
16	RMS frequency	36	Wavelet packet feature 04
17	Dynamic range	37	Wavelet packet feature 05
18	Interquartile range	38	Wavelet packet feature 06
19	Maximum slope	39	Wavelet packet feature 07
20	Zero-crossing rate	40	Wavelet packet feature 08

a year. Each instance in the dataset contains 120 original features. Additionally, the factory occasionally extracts 40 features from the original features in the instances, as shown in Table 3. Based on this, corresponding operations are performed on the collected instances. After preprocessing, a dataset is formed containing 36,333 normal instances and 11,596 fault instances across two different feature spaces. The dataset with 120 original features is used as D_{ori} , and its feature space is P . The dataset containing 40 extracted features is used as D_{new} , and its feature space is C . Moreover, due to uncertainties arising from sensor damage, addition, replacement, and the removal of certain monitoring signals due to their weak representation, the features carried by instances in HRS are often variable. These variations make HRS unpredictable when feature evolution occurs.

For both synthetic and real-world HRS data, the data stream follows a one-pass setting during the experiments, where each instance in the dataset appears only once. Detailed information will be provided in the subsequent experiment.

To enhance the applicability of the model, consistent parameter values are set across different datasets: $B = 50$, $\lambda = 0.99$, $\sigma = 0.5$, and $\rho = 0.5$. These values are determined through grid search, considering the computational efficiency of the model. The experiments are conducted from five aspects: I. Feature Evolution Data Streams with Overlapping Periods, II. Feature Evolution Data Streams without Overlapping Periods, III. Ablation Study, IV. Parameter Study, and V. Time Analysis.

4.2. Experiment I: Feature evolution data streams with overlapping periods

In this section, experiments on feature evolution data streams with overlapping periods are conducted using the proposed approach. Detailed information about the approaches and settings used for comparison is provided, and the experimental results are analyzed.

4.2.1. Experimental setting

In this scenario, the primary comparison is made with FESL and PUF. Following the approaches described in FESL and PUF, the dataset is divided into two stages: the first stage (T1) and the second stage (T2), each containing 50% of the total instances in the dataset. T1 consists solely of all the features from P , and the model is trained. T2 consists solely of all the features from C , and the model is tested using the prediction-while-training method. During the transition from T1 to T2, there exists an overlapping period where both P and C coexist. The number of instances during this overlapping period is set to 50, 100, and 150, respectively. Additionally, for the case where the overlap period is incomplete, as described in PUF, each row of

the remaining feature row in P is randomly and uniformly discarded during the overlapping period. Notably, FESL-c and FESL-s can only run when the overlap period is complete. The experimental conditions are kept consistent across all methods. Ten independent experiments are conducted to evaluate the performance of the proposed method under two scenarios.

4.2.2. Comparisons with state-of-the-arts

The proposed method is compared with six state-of-the-art baselines. NOGD [10] resets the model when the feature space changes. ROGD-f [10] leverages the model trained on P to predict data recovered from C , without updating the model. ROGD-u [10] adopts the same recovery strategy as ROGD-f but updates the model using the recovered data. FESL-c [10] maps C to P and employs ensemble learning. FESL-s [10] also adopts an ensemble approach but dynamically selects the best-performing model instead of combining all models. PUF [6] extends FESL to address non-overlapping feature periods via matrix completion.

4.2.3. Experimental results

Table 4 shows the classification performance of all methods on 14 UCI datasets and the real-world HRS dataset. Whether in the fully overlapping or non-overlapping periods, the classification accuracy of OLUFE significantly outperforms other models. This is due to OLUFE's use of kernel-based nonlinear methods, which enable the model to better capture the nonlinear relationships between features. Furthermore, the Nemenyi test is conducted based on the results in Table 4. In the "P" setting, $k = 7$, $n = 15$, and $q_{0.05} = 2.948$. In the "U" setting, $k = 5$, $n = 15$, and $q_{0.05} = 2.948$. The results of the Nemenyi test are shown in Table 5. The findings indicate that OLUFE ranks the highest, further validating its superior performance on feature evolution data streams with overlapping periods.

Fig. 2 shows the classification error rate trends of all methods on feature evolution data streams with complete overlapping periods. The smaller the classification error rate, the better the performance. The following observations can be made. (i) The NOGD curve exhibits a rapid decrease in all datasets. This is because the NOGD learns better and better with more data. (ii) On all datasets, the curves of ROGD-u, ROGD-f, FESLc, and FESL-s all demonstrate poor trends. This is primarily attributed to the one-pass setting employed in the experiment, resulting in less-than-ideal mapping outcomes of their feature spaces. Moreover, as ROGD-f is fixed, it performs the poorest. (iii) PUF shows no significant disparity from the best results of NOGD, ROGD-f, ROGD-u, FESL-c, and FESL-s on the majority of datasets. This is because it consistently adheres to the optimal model among them. (iv)

Table 4Experimental results for 15 datasets on feature evolution data streams with overlapping periods (average accuracy $\pm$ standard deviation).

Dataset	Setting	NOGD	ROGD-f	ROGD-u	FESL-c	FESL-s	PUFE	OLUFE
HTRU2	P	.830 $\pm$.019•	.669 $\pm$.060•	.678 $\pm$.057•	.769 $\pm$.058•	.762 $\pm$.044•	.744 $\pm$.063•	.904 $\pm$.009
	U	.830 $\pm$.019•	.547 $\pm$.200•	.723 $\pm$.065•	–	–	.743 $\pm$.067•	.904 $\pm$.009
a8a	P	.699 $\pm$.003•	.503 $\pm$.034•	.529 $\pm$.031•	.663 $\pm$.005•	.663 $\pm$.005•	.694 $\pm$.003•	.759 $\pm$.001
	U	.699 $\pm$.003•	.507 $\pm$.029•	.547 $\pm$.025•	–	–	.696 $\pm$.003•	.759 $\pm$.001
ada	P	.676 $\pm$.007•	.504 $\pm$.045•	.561 $\pm$.025•	.646 $\pm$.018•	.626 $\pm$.013•	.640 $\pm$.038•	.746 $\pm$.011
	U	.676 $\pm$.007•	.520 $\pm$.047•	.577 $\pm$.035•	–	–	.637 $\pm$.043•	.746 $\pm$.011
biodeg	P	.632 $\pm$.021○	.485 $\pm$.071•	.601 $\pm$.046•	.626 $\pm$.031○	.615 $\pm$.029•	.632 $\pm$.024○	.652 $\pm$.039
	U	.632 $\pm$.021○	.485 $\pm$.068•	.600 $\pm$.056•	–	–	.638 $\pm$.029○	.652 $\pm$.039
credit_g	P	.574 $\pm$.021•	.547 $\pm$.053•	.597 $\pm$.038•	.607 $\pm$.036•	.590 $\pm$.032•	.596 $\pm$.036•	.680 $\pm$.027
	U	.574 $\pm$.021•	.513 $\pm$.067•	.589 $\pm$.035•	–	–	.584 $\pm$.026•	.680 $\pm$.027
german	P	.594 $\pm$.024•	.496 $\pm$.055•	.565 $\pm$.032•	.594 $\pm$.026•	.581 $\pm$.027•	.590 $\pm$.021•	.692 $\pm$.024
	U	.594 $\pm$.024•	.495 $\pm$.047•	.553 $\pm$.033•	–	–	.593 $\pm$.026•	.692 $\pm$.024
ijcnn1	P	.546 $\pm$.026•	.513 $\pm$.035•	.542 $\pm$.034•	.552 $\pm$.027•	.544 $\pm$.025•	.551 $\pm$.028•	.903 $\pm$.001
	U	.546 $\pm$.026•	.512 $\pm$.060•	.569 $\pm$.034•	–	–	.549 $\pm$.038•	.903 $\pm$.001
mammography	P	.689 $\pm$.009•	.656 $\pm$.061•	.657 $\pm$.065•	.685 $\pm$.017•	.685 $\pm$.012•	.689 $\pm$.010•	.976 $\pm$.001
	U	.689 $\pm$.009•	.434 $\pm$.146•	.565 $\pm$.127•	–	–	.676 $\pm$.030•	.976 $\pm$.001
mc1	P	.940 $\pm$.003•	.793 $\pm$.069•	.837 $\pm$.053•	.938 $\pm$.004•	.936 $\pm$.004•	.938 $\pm$.004•	.993 $\pm$.001
	U	.940 $\pm$.003•	.654 $\pm$.059•	.707 $\pm$.092•	–	–	.938 $\pm$.005•	.993 $\pm$.001
musk2	P	.591 $\pm$.006•	.526 $\pm$.025•	.549 $\pm$.021•	.602 $\pm$.021•	.569 $\pm$.012•	.568 $\pm$.024•	.847 $\pm$.005
	U	.591 $\pm$.006•	.522 $\pm$.023•	.542 $\pm$.018•	–	–	.584 $\pm$.015•	.847 $\pm$.005
mw1	P	.768 $\pm$.016•	.561 $\pm$.120•	.596 $\pm$.079•	.758 $\pm$.015•	.740 $\pm$.042•	.757 $\pm$.015•	.915 $\pm$.020
	U	.768 $\pm$.016•	.558 $\pm$.105•	.610 $\pm$.074•	–	–	.755 $\pm$.017•	.915 $\pm$.020
oil_spill	P	.879 $\pm$.009•	.578 $\pm$.094•	.722 $\pm$.055•	.870 $\pm$.020•	.850 $\pm$.022•	.847 $\pm$.032•	.956 $\pm$.009
	U	.879 $\pm$.009•	.562 $\pm$.097•	.693 $\pm$.070•	–	–	.865 $\pm$.021•	.956 $\pm$.009
pc2	P	.897 $\pm$.005•	.736 $\pm$.081•	.771 $\pm$.066•	.897 $\pm$.004•	.889 $\pm$.007•	.893 $\pm$.007•	.996 $\pm$.001
	U	.897 $\pm$.005•	.642 $\pm$.092•	.568 $\pm$.299•	–	–	.715 $\pm$.373•	.996 $\pm$.001
svmguide3	P	.588 $\pm$.010•	.510 $\pm$.044•	.601 $\pm$.045•	.612 $\pm$.042•	.602 $\pm$.028•	.610 $\pm$.029•	.692 $\pm$.025
	U	.588 $\pm$.010•	.513 $\pm$.055•	.593 $\pm$.040•	–	–	.595 $\pm$.020•	.692 $\pm$.025
HRS	P	.933 $\pm$.001•	.566 $\pm$.033•	.643 $\pm$.034•	.934 $\pm$.001•	.924 $\pm$.002•	.933 $\pm$.001•	.955 $\pm$.007
	U	.933 $\pm$.001•	.529 $\pm$.041•	.644 $\pm$.059•	–	–	.933 $\pm$.001•	.955 $\pm$.007
win/tie/lose		28/2/0	30/0/0	30/0/0	14/1/0	15/0/0	28/2/0	–

The best results are shown in bold. “P” and “U” represent complete and incomplete overlapping periods, respectively. • /○/○ respectively, indicate that OLUFE is significantly better/tied/worse than the comparison method by the t-test [39] with a confidence level of 0.05.

Table 5

Nemenyi test results for OLUFE and competing methods on feature evolution data streams with overlapping periods.

Setting	NOGD	ROGD-f	ROGD-u	FESL-c	FESL-s	PUFE	OLUFE	CD
P	2.800	6.467	5.200	2.600	4.200	3.467	1.000	2.325
U	2.467	4.867	3.667	–	–	2.800	1.000	1.702

Critical Distance (CD): $CD = q_\alpha \sqrt{(k(k+1))/(6n)}$, where q_α is the critical value, k is the number of algorithms, and n is the number of datasets.

The OLUFE curve consistently surpasses other methods on all datasets. Additionally, as new instances increase, the OLUFE curve exhibits an excellent trend. This observation validates that OLUFE can effectively accommodate the dynamic variations of the feature space in the data stream.

4.3. Experiment II: Feature evolution data streams without overlapping periods

In this section, experiments on feature evolution data streams without overlapping periods are conducted using the proposed approach. Detailed information about the approaches and settings used for comparison is provided, and the experimental results are analyzed.

4.3.1. Experimental setting

In this scenario, four different single-cycle settings are constructed based on the feature evolution scenarios outlined in Section 3.1, where the dataset is divided into two phases: the first stage (T1) and the second stage (T2), each containing 50% of the total instances in the dataset. Additionally, a multi-cycle setting is conducted, including four distinct feature evolution scenarios, where the dataset is divided into five phases: the first stage (T1), the second stage (T2), the third stage

(T3), the fourth stage (T4), and the fifth stage (T5), containing 20% of the total instances in the dataset. Specifically, in the single-cycle setting, T1 contains all features from P , and T2 is set up as follows: (i) T2 contains all features from both P and C , forming a trapezoidal data stream; (ii) T2 randomly retains some features from P , with no new features emerging; (iii) T2 randomly retains some features from P and merges them with all features from C ; and (iv) T2 contains only the features from C . In the multi-cycle setting, T1 contains all features from P . T2 contains only features from C , with none retained from P . T3 retains a random subset of features from both P and C . T4 contains all features from both P and C . T5 contains only the features from P . Notably, there is no model pre-training stage in the experimental process. All models are predicted and trained directly on the instances. The experimental conditions are kept consistent across all methods. Ten independent experiments are conducted to evaluate the performance of the proposed method.

4.3.2. Comparisons with state-of-the-arts

The proposed method is compared with ten state-of-the-art baselines. NPA [10] is a passive learning algorithm that restarts from scratch when the feature space changes. OLSF [11] is tailored for trapezoidal data streams, integrating new features via vector enhancement and

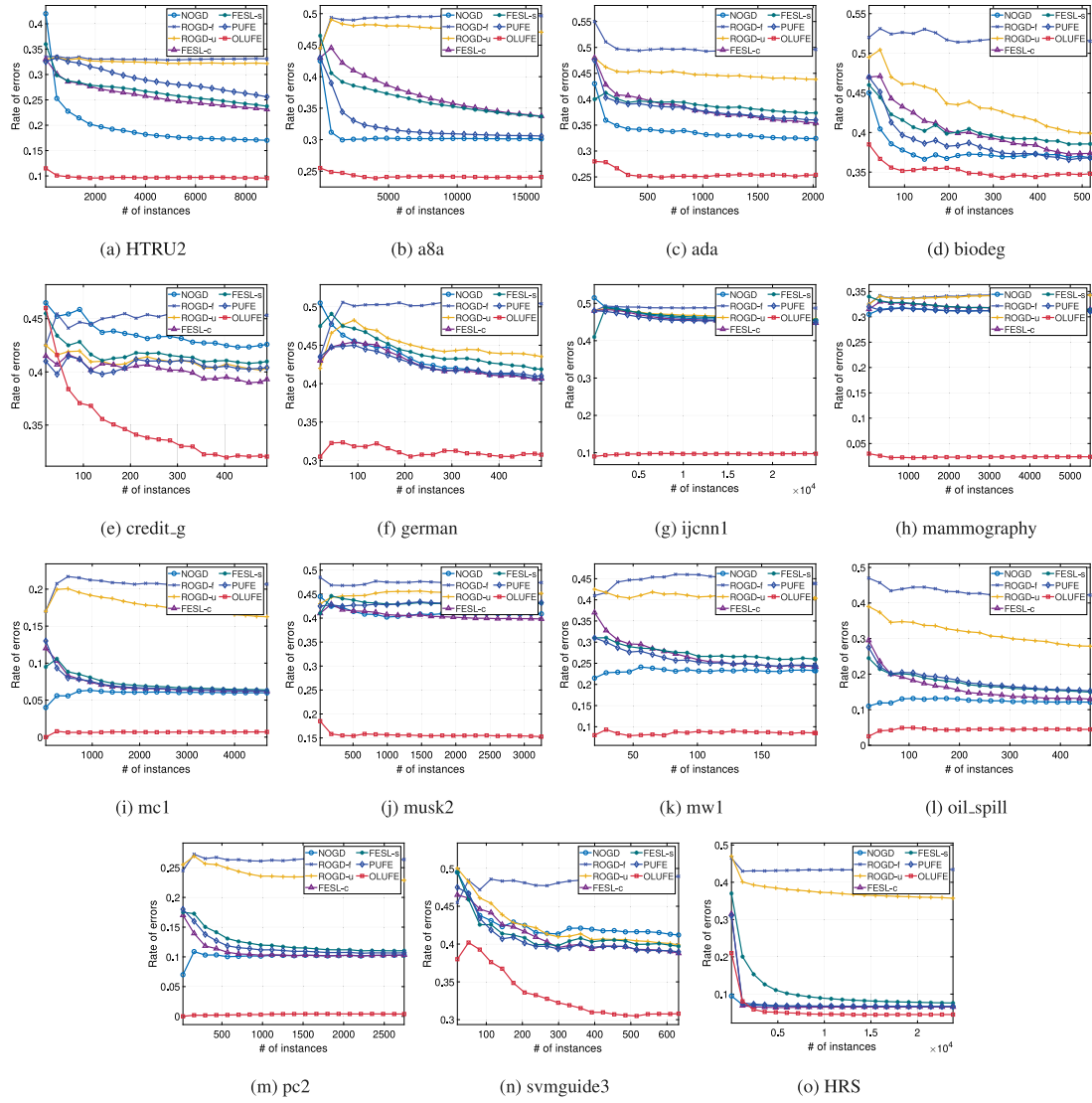


Fig. 2. The average classification error rate trends of OLUFE and six comparison methods on feature evolution data streams with complete overlapping periods.

maintaining sparsity through projection truncation. OPID [12] employs a compression-expansion strategy, compressing vanished feature information into the remaining features and integrating them with new ones for learning. OLVF [19] adapts to feature evolution by projecting the classifier and instances onto their shared subspace and adjusting weights based on projection confidence. GLSC [13] is a generative graph-based method that learns correlations in shared feature spaces. OLI²DS [20] addresses class imbalance and concept drift using empirical risk minimization and a dynamic cost strategy. OLCF [40] handles feature evolution by mapping heterogeneous features into a shared subspace. Exp3-IP [26] incorporates an information-probabilistic feedback graph to guide weight updates. BoKle [28] is an online multi-kernel learning algorithm based on biconvex optimization, enhancing performance via optimized kernel combinations.

4.3.3. Experimental results

Table 6 presents the classification performance of all methods on 14 UCI datasets and the real-world HRS dataset. OLUFE consistently outperforms other methods on most datasets, largely due to its nonlinear learning capability. Compared with BoKle, OLUFE exhibits superior performance, benefiting from its scalability and ability to swiftly adapt to dynamic feature changes. In contrast, OLSF, OPID, and GLSC perform poorly, primarily due to their strong assumptions about feature

evolution, which are not well-suited to the setting where such feature evolution is unpredictable. Moreover, under the multi-cycle setting incorporating four distinct feature evolution scenarios, OLUFE consistently achieves the best performance across all datasets. This robustness is attributed to its adaptive mechanism for expanding and adjusting sub-models, which prioritizes accurate ones during expansion to enhance overall learning performance and ensure long-term reliability. Additionally, the Nemenyi test, as shown in Table 7, is conducted on the data, where $k = 11$, $n = 15$, and $q_{0.05} = 2.948$. The results indicate that OLUFE ranks the highest, further validating its effectiveness in handling data streams with unpredictable feature evolution.

Fig. 3 shows the classification error rate trends of all methods on the feature evolution data stream without overlapping periods under the multi-cycle setting including four different feature evolution scenarios. The following observations can be made: (i) The curves for NOGD and NPA show a relatively stable downward trend. This is because they continuously optimize the classifier with the arrival of new data. Even when feature evolution occurs, requiring the model to restart training, they can still effectively leverage instances with new features for learning. (ii) After feature evolution, the curves for OLVF, OLCF, and Exp-IP exhibit minimal fluctuations and stabilize faster. This is because both OLVF and OLCF effectively utilize old features for prediction and quickly adapt to new ones, while Exp-IP demonstrates

Table 6

Experimental results for 15 datasets on feature evolution data streams without overlapping periods (average accuracy $\pm$ standard deviation).

Dataset	Setting	NOGD	NPA	OLSF	OPID	OLVF	GLSC	OLI ² DS	OLCF	Exp3-IP	BoKle	OLUFE
HTRU2	T	.885 ± .002	.915 ± .001	.875 ± .011	.739 ± .003	.951 ± .001	.883 ± .005	.886 ± .003	.952 ± .001	.913 ± .003	.872 ± .107	.973 ± .004
	I	.881 ± .002	.917 ± .001	.876 ± .010	.869 ± .005	.949 ± .001	.856 ± .008	.885 ± .003	.961 ± .001	.906 ± .001	.902 ± .014	.973 ± .001
	P	.875 ± .013	.890 ± .015	.853 ± .027	.744 ± .067	.924 ± .022	.846 ± .017	.860 ± .018	.935 ± .008	.886 ± .015	.875 ± .095	.953 ± .014
	N	.860 ± .003	.868 ± .002	.813 ± .012	.738 ± .003	.903 ± .002	.770 ± .004	.838 ± .004	.929 ± .002	.875 ± .002	.882 ± .074	.942 ± .005
	A	.854 ± .003	.882 ± .002	.833 ± .011	.825 ± .004	.929 ± .002	.829 ± .003	.853 ± .003	.940 ± .002	.887 ± .002	.874 ± .105	.962 ± .003
a8a	T	.712 ± .001	.727 ± .001	.710 ± .004	.658 ± .003	.741 ± .001	.599 ± .004	.724 ± .001	.719 ± .002	.705 ± .001	.668 ± .060	.759 ± .001
	I	.717 ± .001	.721 ± .001	.710 ± .003	.679 ± .002	.741 ± .003	.572 ± .003	.722 ± .001	.721 ± .002	.703 ± .002	.668 ± .060	.767 ± .001
	P	.709 ± .003	.707 ± .004	.695 ± .010	.630 ± .025	.731 ± .006	.579 ± .008	.715 ± .002	.722 ± .004	.678 ± .009	.668 ± .060	.759 ± .001
	N	.707 ± .001	.702 ± .002	.686 ± .006	.649 ± .003	.716 ± .001	.582 ± .005	.712 ± .001	.726 ± .002	.683 ± .002	.668 ± .060	.759 ± .001
	A	.700 ± .002	.708 ± .002	.699 ± .003	.663 ± .002	.725 ± .002	.582 ± .009	.714 ± .001	.717 ± .002	.688 ± .002	.668 ± .060	.763 ± .003
ada	T	.722 ± .004	.724 ± .003	.631 ± .010	.642 ± .005	.729 ± .007	.723 ± .003	.726 ± .002	.733 ± .008	.714 ± .003	.725 ± .024	.773 ± .010
	I	.734 ± .004	.724 ± .005	.629 ± .012	.679 ± .006	.725 ± .005	.695 ± .006	.729 ± .004	.744 ± .005	.714 ± .007	.713 ± .022	.776 ± .006
	P	.717 ± .006	.704 ± .011	.598 ± .025	.616 ± .026	.721 ± .007	.658 ± .022	.714 ± .008	.729 ± .006	.684 ± .012	.714 ± .025	.758 ± .009
	N	.702 ± .004	.684 ± .007	.600 ± .026	.641 ± .005	.704 ± .006	.660 ± .009	.695 ± .007	.728 ± .005	.676 ± .006	.702 ± .026	.764 ± .003
	A	.708 ± .005	.688 ± .009	.629 ± .011	.700 ± .012	.711 ± .005	.682 ± .004	.695 ± .006	.712 ± .007	.677 ± .009	.712 ± .027	.760 ± .018
biodeg	T	.737 ± .006	.750 ± .006	.736 ± .011	.652 ± .011	.750 ± .011	.719 ± .009	.756 ± .006	.766 ± .009	.744 ± .007	.658 ± .005	.800 ± .013
	I	.745 ± .006	.723 ± .009	.736 ± .012	.709 ± .009	.748 ± .004	.684 ± .014	.742 ± .006	.766 ± .006	.721 ± .010	.659 ± .004	.783 ± .011
	P	.723 ± .015	.722 ± .014	.708 ± .027	.652 ± .029	.727 ± .010	.674 ± .030	.732 ± .015	.727 ± .021	.704 ± .020	.660 ± .006	.750 ± .022
	N	.703 ± .011	.698 ± .011	.659 ± .009	.651 ± .012	.717 ± .005	.612 ± .014	.711 ± .008	.721 ± .006	.685 ± .009	.656 ± .007	.745 ± .016
	A	.704 ± .008	.724 ± .006	.683 ± .011	.675 ± .022	.731 ± .012	.673 ± .013	.730 ± .008	.715 ± .015	.713 ± .007	.659 ± .005	.767 ± .024
credit_g	T	.642 ± .010	.654 ± .010	.652 ± .008	.582 ± .018	.624 ± .012	.646 ± .019	.665 ± .008	.659 ± .009	.662 ± .008	.686 ± .020	.696 ± .018
	I	.658 ± .014	.641 ± .013	.652 ± .007	.600 ± .013	.618 ± .016	.592 ± .009	.657 ± .012	.666 ± .007	.639 ± .013	.689 ± .015	.698 ± .012
	P	.626 ± .018	.620 ± .023	.614 ± .035	.575 ± .016	.617 ± .017	.607 ± .023	.633 ± .026	.639 ± .026	.619 ± .038	.689 ± .014	.676 ± .034
	N	.615 ± .014	.609 ± .010	.609 ± .018	.580 ± .013	.610 ± .009	.598 ± .016	.611 ± .008	.634 ± .007	.602 ± .009	.686 ± .021	.689 ± .015
	A	.618 ± .017	.628 ± .015	.628 ± .015	.537 ± .016	.613 ± .016	.577 ± .015	.641 ± .011	.634 ± .009	.628 ± .015	.686 ± .019	.686 ± .020
german	T	.644 ± .016	.667 ± .011	.641 ± .017	.583 ± .009	.632 ± .012	.532 ± .042	.670 ± .006	.634 ± .026	.656 ± .011	.648 ± .066	.691 ± .020
	I	.648 ± .012	.658 ± .011	.640 ± .016	.601 ± .020	.634 ± .011	.524 ± .061	.668 ± .009	.657 ± .008	.650 ± .010	.656 ± .056	.692 ± .006
	P	.639 ± .016	.631 ± .021	.606 ± .035	.565 ± .022	.631 ± .015	.518 ± .046	.633 ± .022	.615 ± .019	.617 ± .025	.654 ± .060	.696 ± .009
	N	.627 ± .017	.634 ± .011	.623 ± .012	.583 ± .011	.624 ± .009	.528 ± .031	.634 ± .011	.630 ± .014	.629 ± .011	.650 ± .065	.694 ± .009
	A	.612 ± .009	.653 ± .011	.625 ± .011	.539 ± .015	.623 ± .015	.581 ± .037	.657 ± .009	.615 ± .013	.636 ± .011	.637 ± .081	.686 ± .024
ijcnn1	T	.680 ± .003	.692 ± .003	.593 ± .011	.568 ± .002	.637 ± .002	.587 ± .006	.677 ± .002	.713 ± .001	.684 ± .002	.897 ± .019	.902 ± .001
	I	.682 ± .002	.687 ± .002	.590 ± .010	.637 ± .002	.637 ± .001	.568 ± .005	.674 ± .002	.715 ± .002	.675 ± .003	.898 ± .016	.889 ± .001
	P	.662 ± .015	.645 ± .016	.553 ± .028	.545 ± .009	.624 ± .014	.554 ± .016	.631 ± .017	.700 ± .012	.634 ± .013	.893 ± .032	.901 ± .002
	N	.635 ± .003	.616 ± .004	.500 ± .025	.560 ± .002	.606 ± .002	.548 ± .004	.612 ± .003	.688 ± .001	.609 ± .004	.896 ± .022	.902 ± .001
	A	.637 ± .002	.647 ± .003	.586 ± .010	.607 ± .002	.623 ± .002	.575 ± .007	.634 ± .003	.692 ± .002	.641 ± .004	.897 ± .020	.898 ± .008
mammography	T	.700 ± .003	.693 ± .003	.673 ± .016	.609 ± .006	.741 ± .003	.687 ± .004	.603 ± .004	.811 ± .004	.720 ± .004	.950 ± .002	.977 ± .000
	I	.695 ± .003	.695 ± .003	.672 ± .015	.634 ± .006	.740 ± .004	.571 ± .017	.644 ± .010	.798 ± .004	.726 ± .009	.950 ± .002	.977 ± .000
	P	.698 ± .004	.684 ± .009	.673 ± .019	.558 ± .028	.715 ± .020	.682 ± .016	.607 ± .022	.778 ± .021	.718 ± .008	.950 ± .002	.977 ± .000
	N	.694 ± .003	.690 ± .003	.658 ± .009	.608 ± .006	.665 ± .002	.685 ± .004	.609 ± .003	.746 ± .005	.716 ± .003	.950 ± .002	.977 ± .000
	A	.669 ± .009	.668 ± .007	.669 ± .020	.693 ± .008	.710 ± .011	.668 ± .005	.593 ± .009	.764 ± .013	.706 ± .007	.950 ± .002	.977 ± .000
mc1	T	.875 ± .002	.861 ± .002	.854 ± .010	.890 ± .003	.918 ± .004	.728 ± .002	.722 ± .008	.906 ± .004	.846 ± .003	.946 ± .001	.993 ± .000
	I	.852 ± .002	.850 ± .002	.832 ± .013	.865 ± .003	.914 ± .003	.828 ± .002	.824 ± .006	.920 ± .003	.852 ± .003	.946 ± .001	.993 ± .000
	P	.875 ± .002	.863 ± .003	.857 ± .009	.810 ± .016	.915 ± .007	.799 ± .029	.812 ± .011	.903 ± .006	.851 ± .003	.946 ± .001	.993 ± .000
	N	.875 ± .002	.865 ± .002	.865 ± .003	.890 ± .003	.905 ± .004	.774 ± .002	.808 ± .010	.898 ± .002	.853 ± .003	.946 ± .001	.993 ± .000
	A	.862 ± .010	.861 ± .004	.858 ± .008	.881 ± .008	.913 ± .002	.781 ± .005	.746 ± .010	.900 ± .006	.858 ± .012	.946 ± .001	.993 ± .000
musik2	T	.650 ± .006	.692 ± .011	.684 ± .011	.648 ± .007	.687 ± .003	.501 ± .009	.644 ± .011	.706 ± .008	.687 ± .011	.833 ± .028	.893 ± .005
	I	.672 ± .010	.653 ± .006	.687 ± .012	.649 ± .004	.683 ± .004	.528 ± .008	.639 ± .004	.706 ± .005	.655 ± .003	.835 ± .024	.874 ± .007
	P	.644 ± .007	.646 ± .012	.666 ± .018	.613 ± .024	.666 ± .008	.518 ± .015	.623 ± .009	.685 ± .009	.640 ± .018	.832 ± .030	.866 ± .005
	N	.638 ± .007	.640 ± .011	.648 ± .015	.608 ± .007	.639 ± .007	.539 ± .004	.616 ± .011	.674 ± .008	.640 ± .015	.831 ± .032	.870 ± .003
	A	.637 ± .008	.655 ± .013	.669 ± .014	.560 ± .008	.656 ± .002	.527 ± .009	.630 ± .008	.667 ± .006	.650 ± .010	.832 ± .030	.879 ± .008
mw1	T	.728 ± .010	.714 ± .011	.696 ± .013	.678 ± .028	.679 ± .012	.684 ± .008	.684 ± .009	.694 ± .013	.703 ± .010	.911 ± .009	.920 ± .003
	I	.738 ± .014	.732 ± .012	.689 ± .013	.703 ± .020	.694 ± .020	.718 ± .011	.720 ± .014	.750 ± .019	.733 ± .013	.910 ± .011	.918 ± .005
	P	.729 ± .009	.726 ± .010	.700 ± .013	.671 ± .029	.689 ± .017	.703 ± .013	.705 ± .018	.665 ± .016	.719 ± .012	.909 ± .010	.920 ± .002
	N	.731 ± .008	.723 ± .010	.725 ± .013	.677 ± .025	.695 ± .011	.718 ± .019	.683 ± .059	.700 ± .015	.728 ± .011	.910 ± .010	.921 ± .002
	A	.738 ± .012	.722 ± .013	.695 ± .015	.620 ± .039	.693 ± .016	.695 ± .020	.667 ± .012	.714 ± .017	.715 ± .013	.910 ± .009	.921 ± .002
oil_spill	T	.787 ± .024	.770 ± .020	.628 ± .021	.771 ± .023	.777 ± .019	.702 ± .013	.703 ± .031	.782 ± .025	.744 ± .018	.811 ± .018	.960 ± .003
	I	.789 ± .017	.764 ± .017	.616 ± .022	.725 ± .024	.778 ± .012	.778 ± .007	.742 ± .011	.787 ± .017	.791 ± .008	.811 ± .017	.958 ± .004
	P	.787 ± .022	.781 ± .023	.653 ± .025	.730 ± .036	.774 ± .019	.738 ± .029	.712 ± .037	.790 ± .030	.762 ± .031	.811 ± .018	.956 ± .002
	N	.788 ± .025	.785 ± .021	.689 ± .010	.771 ± .024	.771 ± .019	.761 ± .019	.739 ± .031	.776 ± .031	.776 ± .026	.811 ± .018	.956 ± .001
	A	.784 ± .018	.739 ± .026	.664 ± .019	.789 ± .036	.776 ± .012	.597 ± .012	.692 ± .030	.757 ± .013	.693 ± .022	.811 ± .018	.958 ± .003
pc2	T	.850 ± .005	.819 ± .002	.827 ± .010	.865 ± .008	.901 ± .004	.718 ± .002	.736 ± .014	.880 ± .005	.809 ± .006	.978 ± .001	.996 ± .000
	I	.825 ± .003	.817 ± .003	.815 ± .011	.848 ± .006	.905 ± .003	.802 ± .003	.801 ± .004	.898 ± .005	.825 ± .007	.978 ± .001	.996 ± .000
	P	.850 ± .005	.820 ± .003	.821 ± .009	.804 ± .040	.892 ± .008	.796 ± .013	.797 ± .011	.876 ± .007	.815 ± .005	.978 ± .001	.996 ± .000
	N	.850 ± .005	.823 ± .002	.835 ± .010	.865 ± .008	.886 ± .003	.802 ± .003	.826 ± .020	.874 ± .004	.816 ± .004	.978 ± .001	.996 ± .000
	A	.835 ± .005	.840 ± .003	.838 ± .007	.900 ± .020	.887 ± .005	.777 ± .003	.770 ± .016	.879 ± .008	.839 ± .006	.978 ± .001	.996 ± .000
svmguide3	T	.661 ± .010	.650 ± .007	.678 ± .030	.653 ± .012	.708 ± .006	.620 ± .006	.669 ± .011	.719 ± .007	.655 ± .009	.730 ± .010	.762 ± .025
	I	.666 ± .011	.642 ± .008	.679 ± .030	.684 ± .010	.698 ± .007	.611 ± .009	.659 ± .013	.731 ± .006	.650 ± .010	.730 ± .009	.747 ± .011
	P	.646 ± .011	.626 ± .016	.651 ± .053	.608 ± .022	.680 ± .021	.610 ± .012	.633 ± .022	.681 ± .035	.628 ± .014	.730 ± .009	.723 ± .026
	N	.636 ± .009	.618 ± .007	.646 ± .038	.639 ± .011	.667 ± .009	.604 ± .008	.620 ± .013	.671 ± .012	.623 ± .015	.730 ± .010	.715 ± .018
	A	.629 ± .009										

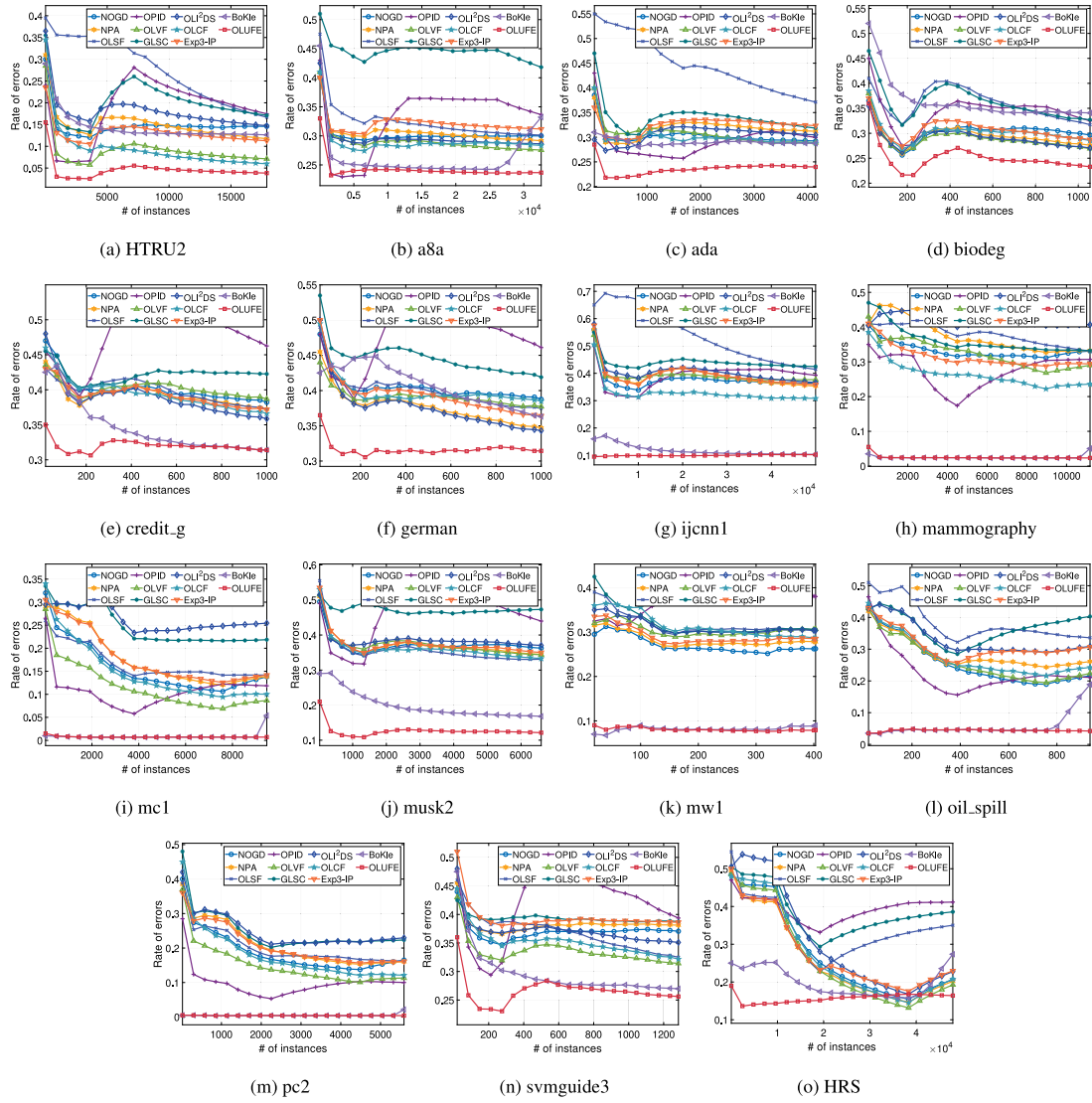


Fig. 3. The average classification error rate trends of OLUFE and seven comparison methods on feature evolution data streams without overlapping periods.

trend, as it effectively captures nonlinear relationships between features through online multi-kernel learning. (v) The OLUFE curve consistently maintains the best trend across all datasets, indicating that OLUFE can promptly detect changes in the feature space and rapidly update the classifier to adapt to the dynamic changes in the data stream.

4.4. Experiment III: Ablation study

Section 3.3 introduces the support vector selection and ensemble prediction strategies of OLUFE. To evaluate their impact on the performance of the algorithm, an ablation study is conducted using the multi-cycle setting with four different feature evolution scenarios, as described in Experiment 4.3. The two variants, OLUFE_NS and OLUFE_NE, represent the versions of OLUFE with the support vector selection and ensemble strategies removed, respectively. The results are shown in Fig. 4.

On most datasets, such as HTRU2, musk2, and HRS, OLUFE demonstrated the best performance. This suggests that the support vector selection and ensemble strategies are crucial for the algorithm's performance. It is important to note that the effectiveness of these strategies depends on the data characteristics. For instance, on the pc2 dataset, OLUFE, OLUFE_NS, and OLUFE_NE produced similar results. On the credit_g, svmguide3, and ijcnn1 datasets, OLUFE's performance was

slightly weaker than OLUFE_NS. This suggests that the algorithm needs to be adaptively adjusted according to the data characteristics, as inappropriate parameters can affect the model components and lead to a reduction in performance.

4.5. Experiment IV: Parameter study

This section conducts experiments using the multi-cycle setting with four different feature evolution scenarios from Experiment 4.3 to investigate further the impact of key parameters in OLUFE on feature evolution data streams. OLUFE includes four key parameters: the sensitivity of support vector selection, ρ ; the support vector budget, B ; the bandwidth of the Gaussian kernel, σ ; and the decay rate controlling the sub-model weights, λ . The values for these parameters are set as follows: $\rho \in \{0.5, 0.7, 0.9, 1.0\}$, $B \in \{10, 20, 50, 100\}$, $\sigma \in \{0.3, 0.5, 0.7, 1.0\}$, and $\lambda \in \{0.9, 0.99, 0.999, 0.9999\}$. During the analysis of each parameter, the other parameters are kept constant, i.e., $B = 50$, $\lambda = 0.99$, $\sigma = 0.5$, and $\rho = 0.5$.

- Parameters ρ : Fig. 5(a) shows the impact of parameter ρ on the prediction accuracy of the model. As ρ changes, the model's performance correspondingly changes. This is because ρ governs the selection of support vectors, which are crucial for the model

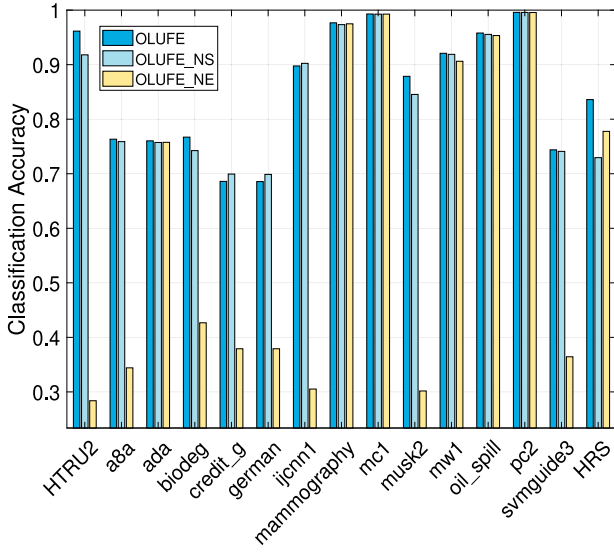


Fig. 4. Ablation study of OLUFE across 15 datasets.

performance. An excessively large ρ may hinder the model from identifying truly representative support vectors, while an excessively small ρ can make it difficult for the model to select support vectors.

- Parameters B : Fig. 5(b) shows the impact of parameter B on the prediction accuracy of the model. On most datasets, increasing B enhances the model performance. However, on some datasets, a larger B may fail to improve the model performance. This is because more support vectors in these datasets may introduce redundant information, reducing the sparsity and generalization of the model. Additionally, a larger B increases the computational complexity of the model. To balance performance and computational efficiency, setting B to 50 can usually provide optimal results.
- Parameters σ : Fig. 5(c) illustrates the effect of parameter σ on the prediction accuracy of the model. σ is a key parameter that directly affects the Gaussian kernel used by the model. When σ is excessively small or large, the model may suffer from underfitting or overfitting, respectively, resulting in a decrease in model performance.
- Parameters λ : Fig. 5(d) demonstrates the impact of parameter λ on the prediction accuracy of the model. It can be seen that λ has a positive impact on the model performance. However, its value must be carefully chosen. When λ is excessively large, the model relies exclusively on the sub-model corresponding to the current feature space, potentially neglecting contributions from other sub-models. Conversely, when λ is excessively small, the model comprehensively considers all sub-models, which may cause the importance of the sub-model corresponding to the current feature space to be ignored.

4.6. Experiment V: Time analysis

Through analysis, the time complexity of OLUFE is $\mathcal{O}(d_t \times B)$. To further evaluate the computational efficiency of OLUFE in practical scenarios, experiments are conducted using the multi-cycle setting with four different feature evolution scenarios from Experiment 4.3. The parameters of OLUFE are set as $B = 50$, $\lambda = 0.99$, $\sigma = 0.5$, and $\rho = 0.5$.

Table 8 presents the runtime (in seconds) for various algorithms. It can be observed that OLUFE requires slightly longer runtime than linear algorithms such as NOGD, NPA, OLSF, and OLVF. This difference

in runtime is due to the scalability of OLUFE. As the number of instances in the data stream increases, the number of support vectors in OLUFE also increases to maintain model performance. However, this also leads to higher computational complexity. In contrast, OLUFE demonstrates a significant advantage compared to nonlinear algorithms like BoKle. This is attributed to the projection-based budget strategy, which ensures that the model's computational complexity does not grow indefinitely. Therefore, the key to balancing model performance and computational efficiency lies in controlling the number of support vectors, which is achieved by adjusting the parameter B .

5. Conclusion

This paper addresses a more realistic scenario of feature evolution data streams by removing restrictive assumptions on feature evolution and proposes a kernel-based online learning algorithm, OLUFE. The algorithm is capable of adapting to unpredictable feature evolution in data streams and effectively handles both the emergence and disappearance of features. The theoretical analysis provides a regret-bound guarantee for the algorithm, and extensive experiments on both public datasets and a real-world HRS dataset demonstrate the effectiveness of the proposed method.

Compared with linear approaches, OLUFE exhibits higher time complexity. Although the use of a support vector selection mechanism helps alleviate this issue, future research should continue to explore more efficient online learning methods for feature evolution data streams, especially in scenarios with strict computational constraints. Moreover, the current version of OLUFE focuses on binary classification tasks, but its framework can be extended to multi-class settings. A straightforward approach is to adopt a one-vs-rest strategy, maintaining an independent set of binary sub-models for each class and aggregating their outputs through voting or probability fusion. Additionally, multi-class kernel support vector machines can be employed to jointly optimize the multi-class weight vectors. Future work will explore the feasibility of such extensions and evaluate their effectiveness in handling multi-class feature evolution data streams.

CRediT authorship contribution statement

Peng Zhang: Writing – review & editing, Writing – original draft, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Hongpeng Yin:** Writing – review & editing, Software, Methodology, Investigation, Funding acquisition, Conceptualization. **Han Zhou:** Writing – review & editing, Validation, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work received partial support from the National Natural Science Foundation of China (Project 62273062), the National Key Research and Development Program Project (Project 2023YFB3308800), the Chongqing Municipality Academician-Led Science and Technology Innovation Guidance Special Project (Project CSTB2023YSZX-JSX0005), and the Major Key Project of PCL (Project PCL2024A05).

Data availability

No data was used for the research described in the article.

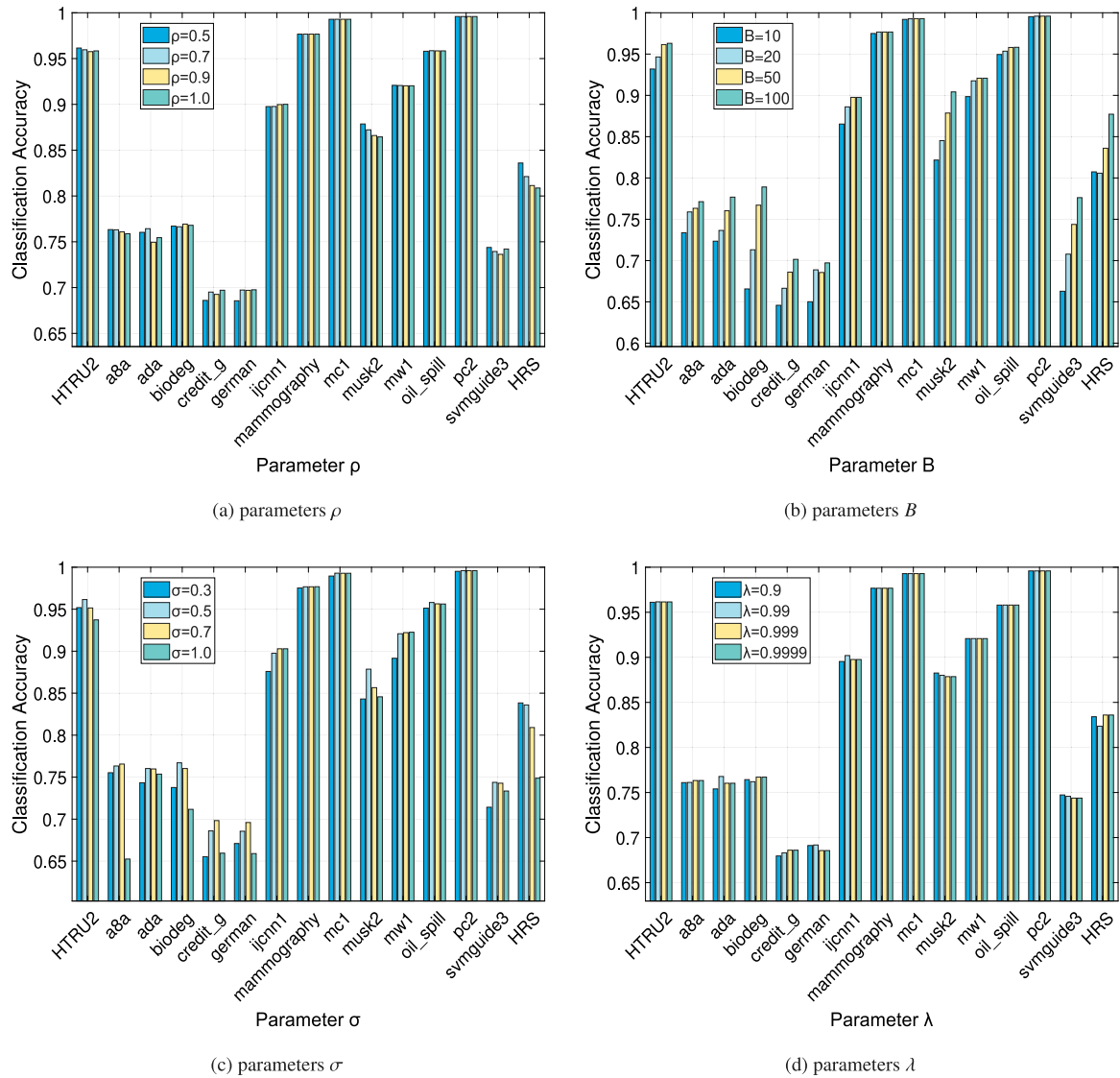


Fig. 5. The impact of parameters ρ , B , σ , and λ on the classification accuracy of OLUFE.

Table 8

Running time comparison of different methods across 15 datasets in the same feature evolution data stream scenario.

Dataset	NOGD	NPA	OLSF	OPID	OLVF	GLSC	OLI ² DS	OLCF	Exp3-IP	BoKle	OLUFE
HTRU2	0.035	0.037	0.042	2.971	0.135	0.226	62.455	2.849	3.280	4.551	0.463
a8a	0.125	0.126	0.277	7.832	0.515	76.955	1056.404	22.695	110.533	53.589	3.631
ada	0.010	0.010	0.018	0.827	0.041	0.617	38.553	1.007	2.692	1.996	0.155
biodeg	0.003	0.003	0.005	0.181	0.011	0.243	11.143	0.260	0.775	0.582	0.043
credit_g	0.002	0.002	0.004	0.190	0.009	0.017	4.262	0.182	0.246	0.270	0.032
german	0.003	0.003	0.006	0.203	0.011	0.227	11.151	0.259	0.778	0.593	0.044
ijcnn1	0.107	0.110	0.184	8.211	0.477	3.217	326.291	9.420	20.121	18.733	1.169
mammography	0.020	0.021	0.035	2.123	0.092	0.084	19.966	1.475	1.020	2.971	0.170
mc1	0.024	0.025	0.031	1.850	0.086	0.563	67.645	1.689	4.138	4.656	0.165
musk2	0.025	0.024	0.065	2.288	0.101	11.194	189.481	4.024	18.861	8.099	0.647
mw1	0.001	0.001	0.002	0.068	0.005	0.058	3.703	0.088	0.251	0.202	0.011
oil_spill	0.002	0.002	0.004	0.184	0.009	0.129	8.485	0.201	0.578	0.808	0.025
pc2	0.012	0.013	0.018	1.083	0.051	0.323	38.450	1.008	2.459	2.318	0.097
svmguide3	0.003	0.003	0.005	0.245	0.012	0.023	5.726	0.231	0.344	0.360	0.041
HRS	0.150	0.147	0.321	12.125	0.554	33.577	919.743	18.020	78.009	55.457	2.584

References

- [1] H. Yu, W. Liu, J. Lu, Y. Wen, X. Luo, G. Zhang, Detecting group concept drift from multiple data streams, *Pattern Recognit.* 134 (2023) 109113.
- [2] H. Guo, Z. Wu, Q. Ren, W. Wang, Accelerating the convergence of concept drift based on knowledge transfer, *Pattern Recognit.* 159 (2025) 111145.
- [3] H. Zhou, H. Yin, X. Deng, Y. Huang, Online harmonizing gradient descent for imbalanced data streams one-pass classification, *IJCAI* (2023) 2468–2475.
- [4] Y. Qin, W. Ju, H. Wu, X. Luo, M. Zhang, Learning graph ode for continuous-time

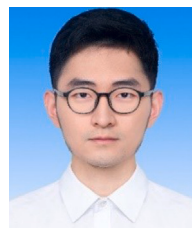
- sequential recommendation, *IEEE Trans. Knowl. Data Eng.* (2024).
- [5] H. Ni, S. Gu, R. Fan, C. Hou, Feature incremental learning with causality, *Pattern Recognit.* 146 (2024) 110033.
 - [6] B.-J. Hou, L. Zhang, Z.-H. Zhou, Prediction with unpredictable feature evolution, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (10) (2021) 5706–5715.
 - [7] H. Zhou, H. Yin, Y. Qin, C. Yuen, Industrial fault diagnosis with incremental learning capability under varying sensory data, *IEEE Trans. Syst. Man Cybern.: Syst.* (2024).
 - [8] M. Zinkevich, Online convex programming and generalized infinitesimal gradient ascent, in: *Proceedings of the 20th International Conference on Machine Learning, icml-03*, 2003, pp. 928–936.
 - [9] Z. Wang, D. Wu, R. Gravina, G. Fortino, Y. Jiang, K. Tang, Kernel fusion based extreme learning machine for cross-location activity recognition, *Inf. Fusion* 37 (2017) 1–9.
 - [10] B.J. Hou, L. Zhang, Z.H. Zhou, Learning with feature evolvable streams, *IEEE Trans. Knowl. Data Eng. PP* (99) (2017).
 - [11] Q. Zhang, P. Zhang, G. Long, W. Ding, C. Zhang, X. Wu, Online learning from trapezoidal data streams, *IEEE Trans. Knowl. Data Eng.* 28 (10) (2016) 2709–2723.
 - [12] C. Hou, Z.-H. Zhou, One-pass learning with incremental and decremental features, *IEEE Trans. Pattern Anal. Mach. Intell.* 40 (11) (2017) 2776–2792.
 - [13] Y. He, B. Wu, D. Wu, E. Beyazit, S. Chen, X. Wu, Toward mining capricious data streams: A generative approach, *IEEE Trans. Neural Netw. Learn. Syst.* 32 (3) (2020) 1228–1240.
 - [14] L. Wang, B. Sun, J. Robinson, T. Jing, Y. Fu, EV-action: Electromyography-vision multi-modal action dataset, in: *2020 15th IEEE International Conference on Automatic Face and Gesture Recognition, FG 2020, IEEE*, 2020, pp. 160–167.
 - [15] J. Zhao, Y. Ling, F. Huang, J. Wang, E.W. See-To, Incremental feature selection for dynamic incomplete data using sub-tolerance relations, *Pattern Recognit.* 148 (2024) 110125.
 - [16] H. Zhou, H. Yin, D. Zhao, L. Cai, Incremental learning and conditional drift adaptation for nonstationary industrial process fault diagnosis, *IEEE Trans. Ind. Inform.* 19 (4) (2022) 5935–5944.
 - [17] Z.-Y. Zhang, P. Zhao, Y. Jiang, Z.-H. Zhou, Learning with feature and distribution evolvable streams, *Int. Conf. Mach. Learn.* (2020) 11317–11327.
 - [18] H.-J. Ye, D.-C. Zhan, Y. Jiang, Z.-H. Zhou, Rectify heterogeneous models with semantic mapping, *Int. Conf. Mach. Learn.* (2018) 5630–5639.
 - [19] E. Beyazit, J. Alagurajah, X. Wu, Online learning from data streams with varying feature spaces, *Proc. AAAI Conf. Artif. Intell.* 33 (01) (2019) 3232–3239.
 - [20] D. You, J. Xiao, Y. Wang, H. Yan, D. Wu, Z. Chen, L. Shen, X. Wu, Online learning from incomplete and imbalanced data streams, *IEEE Trans. Knowl. Data Eng.* 35 (10) (2023) 10650–10665.
 - [21] H. Zhou, H. Yin, Y. Qin, C. Yuen, Industrial fault diagnosis with incremental learning capability under varying sensory data, *IEEE Trans. Syst. Man Cybern.: Syst.* (2024).
 - [22] F. Liu, X. Huang, Y. Chen, J.A. Suykens, Random features for kernel approximation: A survey on algorithms, theory, and beyond, *IEEE Trans. Pattern Anal. Mach. Intell.* 44 (10) (2021) 7128–7148.
 - [23] S. Hong, J. Chae, Communication-efficient randomized algorithm for multi-kernel online federated learning, *IEEE Trans. Pattern Anal. Mach. Intell.* 44 (12) (2021) 9872–9886.
 - [24] A. Alam, A. Malhotra, I.D. Schizas, Online kernel-based clustering, *Pattern Recognit.* 158 (2025) 111009.
 - [25] A. Ferrari, C. Richard, A. Bouchikhi, Online change-point detection with kernels, *Pattern Recognit.* 133 (2023) 109022.
 - [26] P.M. Ghari, Y. Shen, Online learning with uncertain feedback graphs, *IEEE Trans. Neural Netw. Learn. Syst.* (2023).
 - [27] S. Hong, J. Chae, Distributed online learning with multiple kernels, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (3) (2021) 1263–1277.
 - [28] S. Hong, Online multikernel learning method via online biconvex optimization, *IEEE Trans. Neural Netw. Learn. Syst.* (2023).
 - [29] K.-S. Jun, A. Bhargava, R. Nowak, R. Willett, Scalable generalized linear bandits: Online computation and hashing, *Adv. Neural Inf. Process. Syst.* (2004).
 - [30] T. Kannan, H. Hoffmann, Budget rnns: Multi-capacity neural networks to improve in-sensor inference under energy budgets, in: *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium, RTAS, IEEE*, 2021, pp. 143–156.
 - [31] V. Cerone, E. Fadda, D. Regruto, A kernel-based approach to errors-in-variables identification of stable multivariable linear systems, *IEEE Trans. Autom. Control* (2024).
 - [32] T. Le, T.D. Nguyen, V. Nguyen, D. Phung, Approximation vector machines for large-scale online learning, *J. Mach. Learn. Res.* 18 (1) (2017) 3962–4016.
 - [33] Y. Liao, J. Li, S. Liao, Q. Hu, J. Dang, Ahpatron: A new budgeted online kernel learning machine with tighter mistake bound, *Proc. AAAI Conf. Artif. Intell.* 38 (12) (2024) 13782–13789.
 - [34] F. Qi, J. Li, Y. Zhang, W. Huang, B. Hu, H. Cai, Tensorized multi-kernel clustering via consensus tensor decomposition, *IEEE Trans. Emerg. Top. Comput. Intell.* (2024).
 - [35] J. Li, S. Liao, Learnability in online kernel selection with memory constraint via data-dependent regret analysis, 2024, arXiv preprint arXiv:2407.00916.
 - [36] Y. Shen, T. Chen, G.B. Giannakis, Random feature-based online multi-kernel learning in environments with unknown dynamics, *J. Mach. Learn. Res.* 20 (22) (2019) 1–36.
 - [37] Y. Shen, S. Karimi-Bidhendi, H. Jafarkhani, Distributed and quantized online multi-kernel learning, *IEEE Trans. Signal Process.* 69 (2021) 5496–5511.
 - [38] D. Kwon, J. Park, S. Hong, Tighter regret analysis and optimization of online federated learning, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (12) (2023) 15772–15789.
 - [39] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.* 7 (2006) 1–30.
 - [40] H. Zhou, S. Matsushima, Online learning under capricious feature data streams, in: *Proceedings of the Annual Conference of JSAI, JSAI2023*, 2023, pp. 2D4GS201–2D4GS201.



Peng Zhang received the B.E. degree from College of Automation, Hangzhou Dianzi University, China in 2021. He is currently working towards the Ph.D. degree in the College of Automation, Chongqing University, Chongqing, China. His research interests include stream data mining, fault diagnosis and machine learning.



Hongpeng Yin received the B.E. and Ph.D. degree in School of Automation, Chongqing University in 2003 and 2009. After that, he joined the School of Automation, Chongqing University in 2009. He is a professor with School of Automation, Chongqing University. His current research interests mainly include data-driven process monitoring and fault diagnosis, pattern recognition, and data mining.



Han Zhou received the B.E. degree in automation from Dalian Maritime University, in 2017. He is currently working towards the Ph.D. degree in the College of Automation, Chongqing University, Chongqing, China. His research interests include machine learning, data-driven process monitoring and fault diagnosis.